HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BỘ MÔN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

BÁO CÁO PROJECT 01

HỆ ĐA AGENT HỖ TRỢ SÀNG LỌC NGUY CƠ BỆNH TIỂU ĐƯỜNG

Giảng viên hướng dẫn : **PGS.TS. Trần Đình Quế**

Họ và tên sinh viên : **Trần Hải Long**

Mã sinh viên : **B23DCCN510**

Nhóm học phần : **06**

Hà Nội – 2026

Mục lục

DANH MỤC TỪ VIẾT TẮT **iv**

CHƯƠNG 1. GIỚI THIỆU **1**
1.1 Bài toán và động lực 1
1.2 Mục tiêu và phạm vi 1
1.3 Đóng góp và cấu trúc báo cáo 2

CHƯƠNG 2. HỆ ĐA AGENT **3**
2.1 Agent 3
2.2 Hệ đa agent 3
2.3 Giao tiếp giữa các thành phần 4

CHƯƠNG 3. KIẾN TRÚC TỔNG QUAN **5**
3.1 Sơ đồ của đề bài và hiện thực 5
3.2 Vai trò của các thành phần 7
3.3 Luồng một lượt đánh giá 7
3.4 Công nghệ và lưu trữ 9

CHƯƠNG 4. CÁC THÀNH PHẦN CỦA HỆ THỐNG **10**
4.1 Deep Learning ⇒ Agent 10
4.1.1 Dữ liệu và bài toán 10
4.1.2 Mô hình và huấn luyện 11
4.1.3 Hiệu chuẩn, ngưỡng và phân tầng nguy cơ 14
4.1.4 Từ mô hình thành thành phần của hệ thống 16
4.2 Knowledge Base và RAG 16
4.2.1 Nguồn tri thức và đồ thị Neo4j 16
4.2.2 Truy xuất: ngữ nghĩa kết hợp đồ thị 18
4.3 Bộ luật và Reinforcement Learning 19
4.3.1 Bộ luật an toàn và xử lý xung đột 19
4.3.2 Agent chọn bước hỗ trợ bằng Q-learning 20
4.3.3 Luật ràng buộc chính sách học được 22
4.4 Tư vấn, trợ giúp và cô lập lỗi 22

CHƯƠNG 5. CODE VÀ DEMO **24**
5.1 Cấu trúc mã nguồn 24
5.2 Các hàm chính 25
5.3 Cách chạy 26

5.4 Demo: giao diện web 26
5.5 Ví dụ ba lượt đánh giá 30
5.6 Kiểm thử 31
5.7 Đánh giá 32
5.8 Hạn chế và hướng phát triển 33

KẾT LUẬN **34**

TÀI LIỆU THAM KHẢO **35**

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
ADA	American Diabetes Association	Hiệp hội Đái tháo đường Hoa Kỳ
AP	Average Precision	Độ chính xác trung bình (diện tích dưới đường PR)
AUC	Area Under the (ROC) Curve	Diện tích dưới đường ROC
BMI	Body Mass Index	Chỉ số khối cơ thể
BRFSS	Behavioral Risk Factor Surveillance System	Hệ thống giám sát yếu tố nguy cơ hành vi (CDC)
CV	Cross-Validation	Kiểm chứng chéo
DL	Deep Learning	Học sâu
ĐTĐ	–	Đái tháo đường
FPG	Fasting Plasma Glucose	Glucose huyết tương lúc đói
HbA1c (A1C)	Glycated Hemoglobin	Huyết sắc tố gắn glucose
HNSW	Hierarchical Navigable Small World	Đồ thị thế giới nhỏ phân cấp
KB	Knowledge Base	Cơ sở tri thức
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MAS	Multi-Agent System	Hệ đa tác tử (hệ đa agent)
MDP	Markov Decision Process	Quá trình quyết định Markov
ML	Machine Learning	Học máy
MLP	Multilayer Perceptron	Mạng nơ-ron nhiều lớp
RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy xuất
RL	Reinforcement Learning	Học tăng cường
ROC	Receiver Operating Characteristic	Đường đặc trưng hoạt động của bộ thu

Danh sách hình vẽ

3.1 Sơ đồ hệ thống theo đề bài, kèm thành phần hiện thực của dự án 5
3.2 Luồng xử lý một yêu cầu đánh giá 8

4.1 Cách chia dữ liệu 60/20/20, phân tầng theo nhãn, `random_state=42` 10
4.2 ROC-AUC của năm mô hình ứng viên (CV, kiểm định, kiểm tra), vẽ từ `metrics.json` 12
4.3 Đường ROC và Precision–Recall của MLP hiệu chuẩn trên tập kiểm tra 13
4.4 Đường hiệu chuẩn 14
4.5 Ma trận nhầm lẫn tại ngưỡng nhị phân 14
4.6 Phân bố xác suất theo nhãn và hai mốc phân tầng (thang log) 15
4.7 Đồ thị tri thức về ĐTĐ (nút tô màu theo nhãn: Condition, RiskFactor, Test, Intervention) 17
4.8 Reward trung bình và tỉ lệ bỏ sót ca cần chuyển tuyến của bốn chiến lược (mô phỏng) 21

5.1 Màn nhập hồ sơ: Bé Tiếp hỏi số đo cơ bản và tự tính BMI 26
5.2 Kết quả: mức nguy cơ, hướng xử trí và câu hỏi bổ sung theo nhóm 27
5.3 Ông Mô giải thích kết quả và biểu đồ tác động của đặc trưng 28
5.4 Cô Chứng trích dẫn bằng chứng; Chú Luật kiểm tra an toàn 29
5.5 Thang chỉ số tham khảo, khuyến nghị của Bạn Gợi và *trace* của lượt xử lý 30

Danh sách bảng

2.1 Các loại agent và hiện thực trong dự án 3
2.2 Một khối làm tất cả so với hệ nhiều thành phần của dự án 4

3.1 Ánh xạ khối trên sơ đồ sang thành phần của hệ thống 6
3.2 Các thành phần, đầu vào và kỹ thuật bên trong 7
3.3 Công nghệ sử dụng 9

4.1 Kết quả năm mô hình (ROC-AUC của CV là trung bình ± độ lệch chuẩn) . . . 12
4.2 Tỉ lệ nhãn 1 trên tập kiểm tra theo mức nguy cơ 15
4.3 Thống kê nút và quan hệ của KB 18
4.4 Các luật hiện có trong `tools/rules.py` 20
4.5 So sánh chiến lược trên 20.000 tập mô phỏng 21
4.6 Các cuộc trao đổi giữa thành phần (`agents/dialogue.py`) 23

5.1 Các hàm và lớp chính 25
5.2 Ba hồ sơ chạy thật qua API 31
5.3 Bộ kiểm thử tự động 32

CHƯƠNG 1. GIỚI THIỆU

1.1. Bài toán và động lực

Đái tháo đường (ĐTĐ, bệnh tiểu đường) là tình trạng glucose trong máu tăng cao kéo dài do cơ thể thiếu insulin hoặc kháng insulin. Phần lớn ca bệnh là ĐTĐ típ 2, tiến triển âm thầm nhiều năm và có thể được làm chậm hoặc phòng ngừa nếu phát hiện sớm từ giai đoạn tiền ĐTĐ [1, 2]. Người dân thường chỉ có vài con số rời rạc (một lần thử đường huyết, cân nặng, huyết áp) và không biết chúng nói lên điều gì.

Một công cụ hỗ trợ sàng lọc cần làm được bốn việc: *dự đoán* nguy cơ từ dữ liệu sức khỏe; *giải thích* dựa trên tri thức y khoa có nguồn; *gợi ý* bước tiếp theo; và *cảnh báo* khi số đo ở mức nguy hiểm. Mỗi việc cần một kỹ thuật khác nhau: học sâu cho dự đoán, cơ sở tri thức và RAG cho giải thích, học tăng cường cho việc chọn bước hỗ trợ, và luật tường minh cho an toàn. Gộp tất cả vào một chương trình làm hệ thống khó kiểm thử và khó tin cậy. Vì vậy dự án tách thành các thành phần chuyên trách, do một Orchestrator điều phối.

1.2. Mục tiêu và phạm vi

Dự án hiện thực hóa sơ đồ do giảng viên đưa ra trên lớp (Hình 3.1, Chương 3) thành một hệ thống chạy được đầu–cuối:

- **DB → Deep Learning → dự đoán, phân loại**: huấn luyện và so sánh năm mô hình học máy/học sâu trên bộ dữ liệu CDC Diabetes Health Indicators (BRFSS 2015, 253.680 hồ sơ, 21 đặc trưng); chọn mạng MLP (PyTorch) đã hiệu chuẩn xác suất; phân tầng nguy cơ thành ba mức Thấp, Trung bình, Cao.
- **Web, sách → Neo4j KB → RAG**: đồ thị tri thức về ĐTĐ (20 thực thể, 35 quan hệ, 21 đoạn bằng chứng từ 5 nguồn) trên Neo4j; các đoạn bằng chứng được nhúng bằng `all-MiniLM-L6-v2` và tìm kiếm theo độ tương đồng trên ChromaDB, kết hợp với liên kết từ đồ thị.
- **Reinforcement learning + rule → Agent**: chính sách Q-learning chọn bước hỗ trợ tiếp theo, luôn nằm dưới quyền ưu tiên của bộ luật an toàn.
- **Tư vấn, trợ giúp**: giải thích đặc trưng, khuyến nghị có trích dẫn nguồn, câu hỏi bổ sung theo mức ảnh hưởng và mô phỏng *"nếu thay đổi"*.
- **Code + Demo**: API FastAPI, giao diện web Next.js, bộ kiểm thử tự động.

Phạm vi. Đây là sản phẩm học tập. Hệ thống phục vụ *sàng lọc* chứ không *chẩn đoán*, không đưa liều thuốc, và mọi kết quả đều nhắc người dùng đi khám bác sĩ. Các con số trong báo cáo được lấy từ `artifacts/`, từ lần chạy kiểm thử và từ ví dụ chạy trực tiếp trên mã nguồn của dự án; các giả định chưa kiểm chứng được nêu rõ.

1.3. Đóng góp và cấu trúc báo cáo

Các đóng góp chính:

- Quy trình huấn luyện có kỷ luật đánh giá: chia 60/20/20 có phân tầng, CV 5 lần, tuning hai mô hình đứng đầu, chọn mô hình và ngưỡng *trên tập kiểm định*, chỉ dùng tập kiểm tra để đánh giá cuối.
- Tách ngưỡng nhị phân và các mức Thấp/Trung bình/Cao nhưng cùng chọn trên kiểm định, để một ca bị gắn cờ dương tính không bao giờ ở mức Thấp.
- Bộ luật an toàn có mã luật, mức ưu tiên và căn cứ; *số đo thực tế thắng dự đoán thống kê*, và mọi xung đột được báo cáo cùng cách giải quyết.
- Truy xuất bằng chứng từ đồ thị tri thức Neo4j kết hợp tìm kiếm ngữ nghĩa, mỗi khẳng định trích dẫn được tới nguồn.
- Policy Q-learning chọn bước hỗ trợ tiếp theo, so sánh với các chiến lược cố định, và ghi rõ phản hồi là *mô phỏng*.
- Cô lập lỗi: lỗi của thành phần tùy chọn được ghi vào `degraded`, có nội dung dự phòng; lỗi bộ luật nghiêng về phía chuyển bác sĩ.

Báo cáo gồm: Chương 2 cơ sở về agent và hệ đa agent; Chương 3 kiến trúc tổng quan; Chương 4 từng thành phần; Chương 5 mã nguồn, demo, kiểm thử và đánh giá; cuối cùng là kết luận.

CHƯƠNG 2. HỆ ĐA AGENT

2.1. Agent

Theo Wooldridge [8], agent (tác tử) là một hệ thống tính toán đặt trong một môi trường, có khả năng *hành động tự chủ* trong môi trường đó để đạt mục tiêu được giao. Điều cốt lõi là vòng lặp quan sát → quyết định → hành động → nhận phản hồi. Một agent thông minh có bốn tính chất [9]: tự chủ, phản ứng, chủ động và xã hội.

Russell và Norvig [10] phân loại agent theo độ phức tạp của chương trình bên trong. Bảng 2.1 đặt các thành phần của dự án vào cách phân loại đó. Cần lưu ý đây là phép ánh xạ tương đối: một số thành phần (như bộ luật) đơn giản hơn một agent đầy đủ và được gọi là "thành phần" thay vì "agent".

Bảng 2.1. Các loại agent và hiện thực trong dự án

Loại	Cách ra quyết định	Trong dự án
Phản xạ theo luật	Nếu điều kiện thì hành động; không học	`SafetyGuard` + `RuleEngine` (9 luật: 5 luật cơ sở, 4 luật xung đột)
Có trạng thái	Duy trì bộ nhớ	`MemoryAgent` lưu dự đoán trong ChromaDB và tìm hồ sơ tương tự
Dựa trên tri thức	Suy luận trên tri thức tường minh	`RetrievalAgent`: KB Neo4j + tìm kiếm ngữ nghĩa
Dựa trên mục tiêu (đơn giản)	Chọn chuỗi bước để đạt mục tiêu	`Orchestrator`: quy trình cố định; riêng việc "thu thập thêm" do policy chọn (tối đa 3 bước)
Agent học	Cải thiện từ dữ liệu, phần thưởng	`RiskModel` (MLP, học có giám sát); `QPolicy` (Q-learning)
Sinh văn bản	Diễn đạt kết quả của các thành phần khác	`ExplanationAgent`, `RecommendationAgent` (LLM tùy chọn; mặc định dùng mẫu câu)

2.2. Hệ đa agent

Hệ đa agent (Multi-Agent System, MAS) gồm nhiều agent tương tác để giải quyết bài toán vượt quá khả năng hoặc hiểu biết của từng agent. Nếu chỉ có một khối làm tất cả, nó phải trộn dữ liệu số, văn bản y khoa, chính sách chọn bước và luật an toàn trong cùng một chỗ: khó kiểm thử, khó thay thế và khó biết kết luận đến từ đâu. Bảng 2.2 so sánh hai cách làm.

Bảng 2.2. Một khối làm tất cả so với hệ nhiều thành phần của dự án

Tiêu chí	Một khối	Nhiều thành phần (dự án)
Độ phức tạp	Cao, trộn nhiều kỹ thuật	Mỗi thành phần một kỹ thuật
Kiểm thử	Khó tách lỗi	Kiểm thử riêng từng thành phần (15 tệp kiểm thử)
An toàn	Luật nằm lẫn trong mô hình	Bộ luật độc lập, luôn ưu tiên hơn dự đoán và policy RL
Chịu lỗi	Một lỗi làm hỏng cả yêu cầu	Lỗi được ghi vào `degraded`, dùng nội dung dự phòng
Chi phí phối hợp	Không có	Cần Orchestrator và định dạng dữ liệu trao đổi thống nhất

Mô hình tổ chức. Dự án dùng kiểu *supervisor*: Orchestrator gọi các thành phần theo thứ tự, truyền kết quả giữa chúng và tổng hợp đầu ra. Không có agent nào gọi trực tiếp agent khác.

2.3. Giao tiếp giữa các thành phần

FIPA-ACL [11] chuẩn hóa thông điệp giữa các agent bằng *performative*. Dự án **không** hiện thực đầy đủ FIPA-ACL và không có bus thông điệp bất đồng bộ. Giao tiếp gồm hai tầng:

- **Dữ liệu có cấu trúc**: các thành phần được Orchestrator gọi như hàm Python và trao đổi từ điển JSON có trường rõ ràng, ví dụ `prediction`, `feature_explanation`, `retrieved_context`, `safety`, `policy`. Đủ cho một quy trình tuần tự và dễ kiểm thử.
- **Typed message khi các kết luận bất đồng**: một tập con nhỏ của FIPA-ACL (`agents/dialogue.py`) gồm bốn performative `challenge`, `accept`, `propose`, `inform`. Khi một luật xung đột kích hoạt, `SafetyGuard` gửi `challenge` tới thành phần bị luật bác bỏ; thành phần đó trả lời bằng hàm phản hồi riêng của mình. Một loại đề xuất, `propose` kèm `action`, được Orchestrator thực hiện thật và kết quả được báo lại bằng `inform` (Mục 4.4).

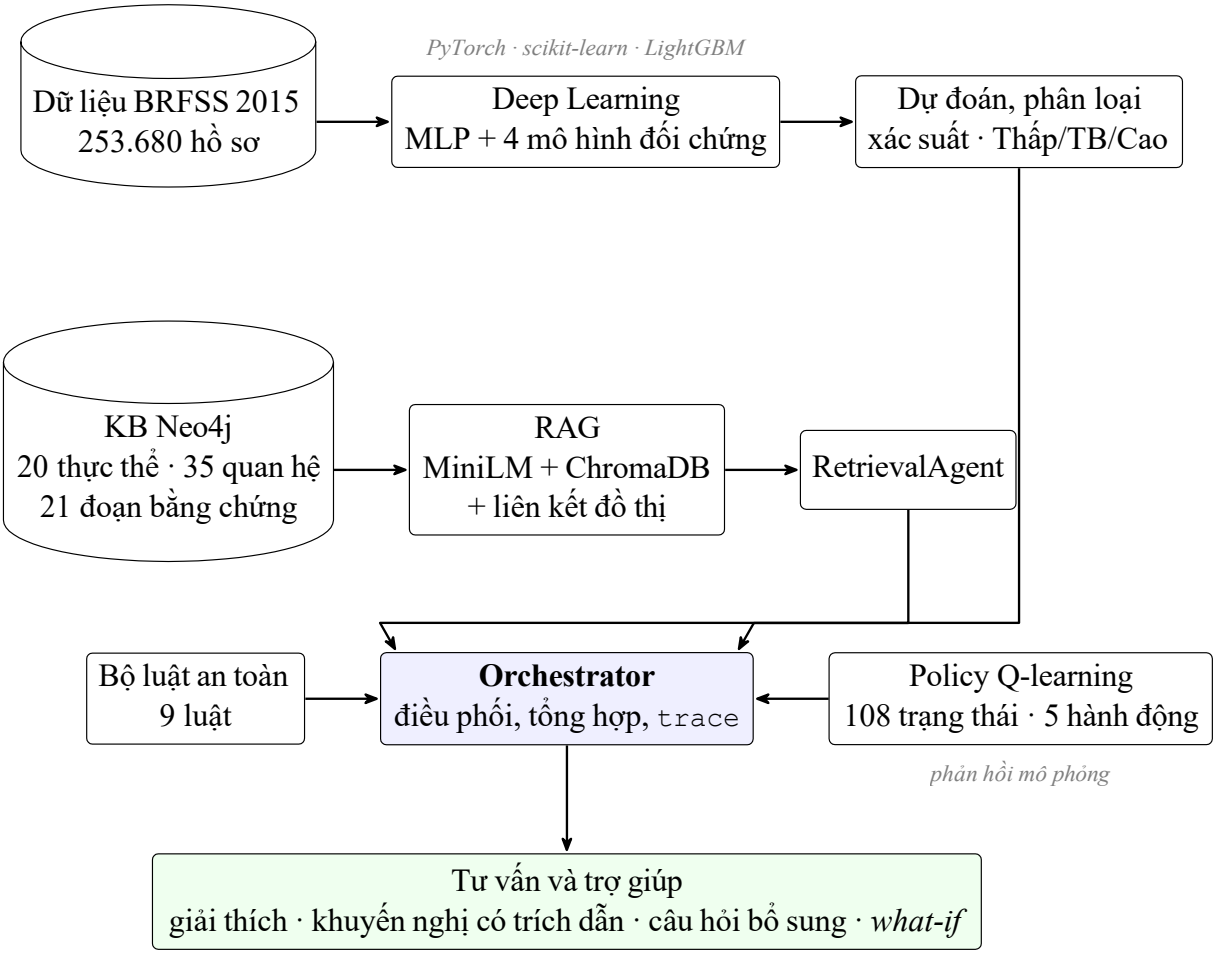
Để vẫn truy vết được "ai đã làm gì", lớp `RunLog` ghi lại mỗi bước (tên thành phần, trạng thái `ok`/`degraded`, thời gian xử lý bằng mili-giây, mô tả ngắn). Trường `trace` trong kết quả API chứa toàn bộ các bước này, cùng danh sách thông điệp `trace.dialogue` giữa các thành phần, và được giao diện hiển thị ở mục "Cả đội đã làm việc thế nào".

Hướng lai LLM. Các mô hình ngôn ngữ lớn thường được dùng làm "bộ não" của agent, kéo theo rủi ro ảo giác lan truyền và chi phí gọi nhiều lần. Dự án chọn hướng lai: mọi quyết định lâm sàng do mô hình đã đánh giá và luật tường minh đảm nhận; LLM (OpenAI, mặc định `gpt-4o-mini`) chỉ *diễn đạt* giải thích và khuyến nghị khi có `OPENAI_API_KEY`. Thiếu khóa hoặc gọi lỗi thì dùng mẫu câu dự phòng và nói rõ điều đó. Nhờ vậy toàn bộ hệ thống chạy được không cần LLM và cho kết quả tái lập.

CHƯƠNG 3. KIẾN TRÚC TỔNG QUAN

3.1. Sơ đồ của đề bài và hiện thực

Hình 3.1 vẽ lại sơ đồ của đề bài kèm thành phần hiện thực. Có ba dòng chảy hội tụ vào Orchestrator: (1) dữ liệu có sẵn đi qua Deep Learning để dự đoán và phân loại; (2) tri thức từ các nguồn y khoa được tạo thành Knowledge Base trên Neo4j và truy xuất bằng RAG; (3) bộ luật và policy reinforcement learning cùng điều khiển việc ra quyết định. Kết quả hội tụ ở đầu ra tư vấn và trợ giúp có kèm bằng chứng.



Hình 3.1. Sơ đồ hệ thống theo đề bài, kèm thành phần hiện thực của dự án

Bảng 3.1 ánh xạ từng khối trên sơ đồ sang mã nguồn.

Bảng 3.1. Ánh xạ khối trên sơ đồ sang thành phần của hệ thống

Khối trên sơ đồ	Thành phần	Hiện thực
DB → Deep learning → Dự đoán, Phân loại	`models/`, `RiskModel`	Huấn luyện bằng `training_pipeline.py`; MLP PyTorch, suy luận bằng NumPy; ngưỡng và mức nguy cơ lưu trong artifact (Mục 4.1)
Neo4j KB	`tools/neo4j_kb.py`, `scripts/load_neo4j_kb.py`	Đọc `data/kb/diabetes_kb.json`, kiểm tra nhất quán rồi nạp (`MERGE`) vào Neo4j; có `File KnowledgeBase` dự phòng (Mục 4.2)
RAG → Agent	`RetrievalAgent`	Nhúng `all-MiniLM-L6-v2`, chỉ mục ChromaDB (cosine), cộng điểm cho đoạn gắn với yếu tố nguy cơ qua đồ thị
Bộ luật	`SafetyGuard`, `RuleEngine`	5 luật cơ sở và 4 luật xung đột (Mục 4.3)
Reinforcement learning	`rl/environment.py`, `rl/policy.py`	Q-learning dạng bảng trên môi trường mô phỏng (Mục 4.3)
Tư vấn, trợ giúp	`ExplanationAgent`, `RecommendationAgent`, `what_if`, `patient_guidance`	Giải thích, khuyến nghị có trích dẫn, mô phỏng "nếu thay đổi", câu hỏi bổ sung (Mục 4.4)

3.2. Vai trò của các thành phần

Bảng 3.2. Các thành phần, đầu vào và kỹ thuật bên trong

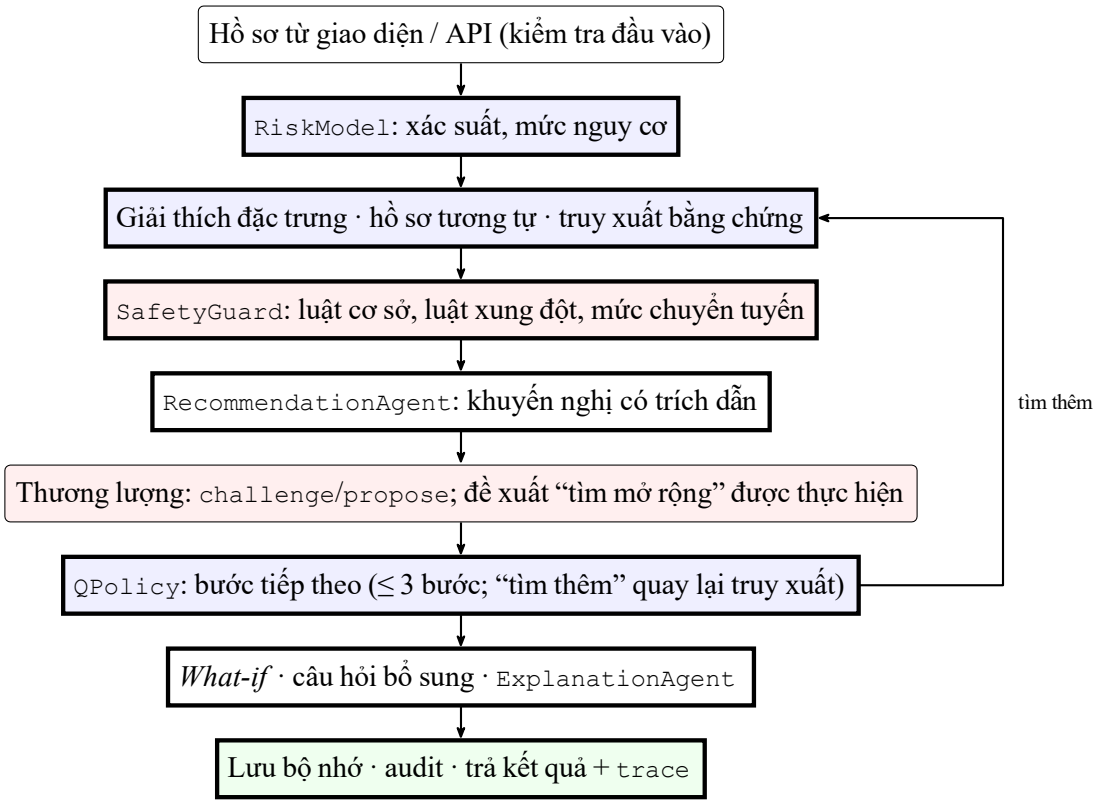
Thành phần	Nhận	Vai trò và kỹ thuật
`Orchestrator`	Hồ sơ đã kiểm tra từ API	Gọi lần lượt các thành phần, truyền kết quả, ghi `trace`, xử lý lỗi, thương lượng giữa các thành phần (`_negotiate`), chạy vòng policy (tối đa 3 bước)
`RiskModel`	Hồ sơ (tuổi, giới, chiều cao, cân nặng, huyết áp, các mục tùy chọn)	Chuẩn hóa về 21 đặc trưng BRFSS, dự đoán xác suất bằng MLP hiệu chuẩn, ghi nhận trường tự điền
`FeatureExplainer`	Hồ sơ	Giải thích bằng phân tích độ nhạy (SHAP chỉ cho mô hình tuyến tính)
`RetrievalAgent`	Hồ sơ, mức nguy cơ, giải thích đặc trưng	Tìm tối đa 3 đoạn bằng chứng (5 khi mở rộng), trả nguồn, URL, độ liên quan, `evidence_status`
`MemoryAgent`	Hồ sơ, xác suất	Lưu dự đoán; tìm hồ sơ tương tự theo khoảng cách số (tuổi, BMI, huyết áp, glucose)
`SafetyGuard`	Hồ sơ, xác suất, mức nguy cơ, trạng thái bằng chứng	Chạy bộ luật, tính điểm chất lượng dữ liệu, xác định mức chuyển tuyến
`QPolicy`	Trạng thái 5 chiều	Chọn bước tiếp theo từ bảng Q
`RecommendationAgent`	Bằng chứng, ca tương tự, kết quả luật, giải thích đặc trưng	Khuyến nghị có trích dẫn `[n]` (LLM tùy chọn; mẫu câu dự phòng)
`ExplanationAgent`	Kết quả cuối của các thành phần	Giải thích bằng lời, chạy sau cùng
`WhatIfSimulator`	Hồ sơ	Tính lại xác suất khi thay đổi một yếu tố có thể thay đổi

3.3. Luồng một lượt đánh giá

Mỗi yêu cầu `POST/v1/predict` đi qua các bước sau (Hình 3.2); các bước tùy chọn được bọc bởi `RunLog.guard` nên lỗi của chúng không làm hỏng cả yêu cầu:

1. API kiểm tra dữ liệu (cần BMI *hoặc* đủ chiều cao và cân nặng, nếu không trả HTTP 422).
2. `RiskModel` dự đoán; xác định nhãn nhị phân và mức nguy cơ theo ngưỡng trong artifact.
3. Tính độ nhạy đặc trưng, tìm hồ sơ tương tự, truy xuất bằng chứng.
4. Đánh giá `evidence_status`, chạy bộ luật, soạn khuyến nghị.
5. *Thương lượng*: nếu có luật xung đột, các thành phần trao đổi thông điệp (Mục 4.4); đề xuất "tìm mở rộng bằng chứng" của `RetrievalAgent` được thực hiện ngay (chạy lại truy xuất, luật và khuyến nghị), tối đa một lần.

6. Nếu có policy: dựng trạng thái (đã tính việc tìm mở rộng ở bước trước), hỏi policy; nếu chọn "tìm thêm bằng chứng" thì chạy lại truy xuất mở rộng, luật và khuyến nghị (tối đa 3 bước).
7. Mô phỏng "nếu thay đổi" (chỉ khi mức chuyển tuyến là theo dõi định kỳ), giải thích cuối, lưu bộ nhớ, ghi audit.



Hình 3.2. Luồng xử lý một yêu cầu đánh giá

3.4. Công nghệ và lưu trữ

Bảng 3.3. Công nghệ sử dụng

Thành phần	Công nghệ	Ghi chú
Học máy / học sâu	scikit-learn 1.6.1, LightGBM, PyTorch	PyTorch chỉ dùng khi huấn luyện MLP; suy luận bằng NumPy nên không cần torch khi chạy
Knowledge Graph	Neo4j [17]	Nạp bằng `MERGE`; Compose khai báo `neo4j: 5-community`
Vector store	ChromaDB [18]	Chỉ mục HNSW, khoảng cách cosine; tên collection băm theo mô hình và nội dung KB
Embedding	`sentence-transformers/all-MiniLM-L6-v2` [13]	Chuẩn hóa đơn vị nên cosine bằng tích vô hướng
LLM (tùy chọn)	OpenAI	Chỉ diễn đạt; thiếu khóa thì dùng mẫu câu
Backend	FastAPI, Pydantic 2, Uvicorn	Xác thực Bearer tùy chọn; audit `logs/decisions.jsonl`
Giao diện	Next.js 15, React 19, TypeScript, Tailwind, Recharts	Có thêm bản Streamlit để thử nhanh
Triển khai	Docker Compose; Terraform cho AWS App Runner	CI GitHub Actions: lint, định dạng, kiểm thử, build web

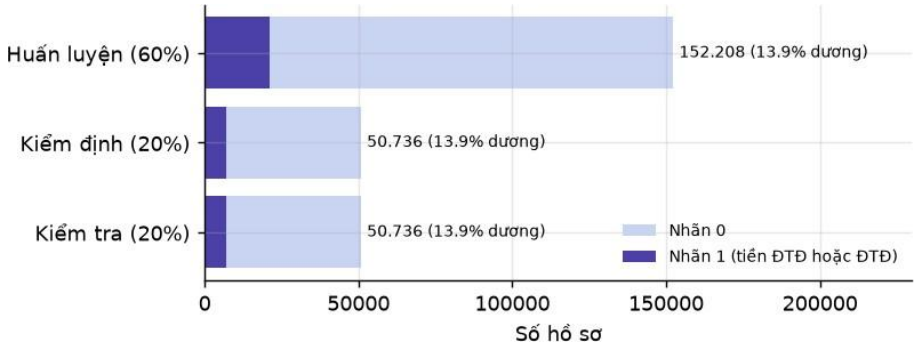
CHƯƠNG 4. CÁC THÀNH PHẦN CỦA HỆ THỐNG

4.1. Deep Learning ⇒ Agent

4.1.1. Dữ liệu và bài toán

Khối "DB có sẵn" là bộ *CDC Diabetes Health Indicators* (BRFSS 2015, bản Kaggle/UCI [7]): **253.680** hồ sơ, 21 đặc trưng (huyết áp cao, mỡ máu cao, BMI, hút thuốc, đột quỵ, bệnh tim, vận động, ăn trái cây và rau, uống nhiều rượu, tình trạng bảo hiểm, sức khỏe tự đánh giá, số ngày sức khỏe thể chất/tinh thần kém, khó đi lại, giới, nhóm tuổi, học vấn, thu nhập). Nhãn `Diabetes_binary` bằng 0 là chưa mắc, bằng 1 là *tiền ĐTĐ hoặc ĐTĐ* (gộp chung); tỉ lệ nhãn 1 là 13,9%. Đây là dữ liệu **tự khai** của người Hoa Kỳ, nên nhãn có thể bỏ sót người mắc chưa được chẩn đoán.

Glucose **không phải** đặc trưng của mô hình: nó được dùng ở bộ luật và phần tư vấn. Đây là lựa chọn có chủ đích, vì mô hình trả lời câu hỏi "với hồ sơ lối sống này, nguy cơ cao đến mức nào", còn số đo đường huyết là căn cứ chẩn đoán do luật kiểm tra. Hai góc nhìn độc lập này là nguyên liệu cho việc đối chiếu (Mục 4.3).



Hình 4.1. Cách chia dữ liệu 60/20/20, phân tầng theo nhãn, `random_state=42`

Cell 1 của `training.ipynb`: khởi tạo, nạp `metrics.json`, định dạng số kiểu Việt Nam

```
import json, sys
from pathlib import Path
import numpy as np, pandas as pd, joblib
import matplotlib.pyplot as plt

ROOT = Path.cwd()
if not (ROOT / "models").exists():
    ROOT = ROOT.parent
sys.path.insert(0, str(ROOT))
OUT = ROOT / "report" / "images"
TAB = ROOT / "report" / "tables"
OUT.mkdir(parents=True, exist_ok=True); TAB.mkdir(parents=True, exist_ok=True)
plt.rcParams.update({"figure.dpi": 130, "font.size": 10, "axes.grid": True, "grid.alpha": .3,
```

```
                    "axes.spines.top": False, "axes.spines.right": False})
metrics = json.loads((ROOT / "artifacts" / "metrics.json").read_text(encoding="utf-8"))
vn = lambda x, d=4: f"{x:.{d}f}".replace(".", ",")      # số thập phân kiểu Việt Nam
print(metrics["model_version"], metrics["rows_total"], metrics["selected_threshold"])
```

Cell 2 của `training.ipynb`: chia dữ liệu và vẽ Hình 4.1

```
from sklearn.model_selection import train_test_split
from models.training_pipeline import FEATURE_NAMES, TARGET_COLUMN, RANDOM_STATE, TEST_SIZE,
↪  VALIDATION_SIZE

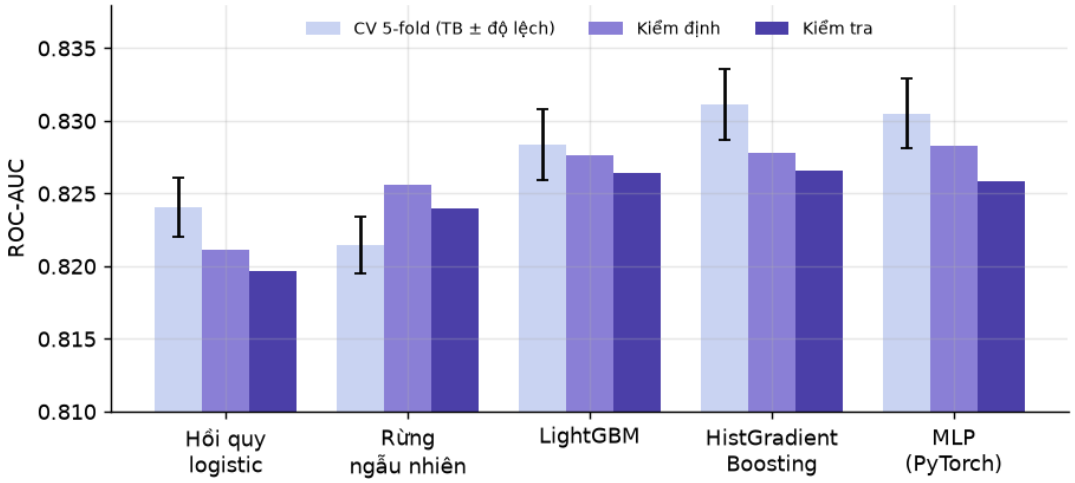
df = pd.read_csv(ROOT / "data" / "diabetes_binary_health_indicators_BRFSS2015.csv")
X, y = df[FEATURE_NAMES], df[TARGET_COLUMN].astype(int)
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE,
↪  stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=VALIDATION_SIZE,
↪  random_state=RANDOM_STATE, stratify=y_dev)
print(len(X_train), len(X_val), len(X_test), "| positive rate:", round(y.mean(), 4))
assert (len(X_train), len(X_val), len(X_test)) == (metrics["rows_train"], metrics["rows_validation"],
↪  metrics["rows_test"])

fig, ax = plt.subplots(figsize=(6.4, 2.6))
sizes = [len(X_train), len(X_val), len(X_test)]
pos = [y_train.sum(), y_val.sum(), y_test.sum()]
names = ["Huấn luyện (60%)", "Kiểm định (20%)", "Kiểm tra (20%)"]
ax.barh(names[::-1], sizes[::-1], color="#c9d3f2", label="Nhãn 0")
ax.barh(names[::-1], pos[::-1], color="#4b3fa8", label="Nhãn 1 (tiền ĐTĐ hoặc ĐTĐ)")
for i, (s, p) in enumerate(zip(sizes[::-1], pos[::-1])):
    ax.text(s + 2000, i, f"{s:,} ({p/s:.1%} dương)".replace(",", "."), va="center", fontsize=8)
ax.set_xlim(0, 230000); ax.set_xlabel("Số hồ sơ"); ax.legend(loc="lower right", fontsize=8, frameon=False)
fig.tight_layout(); fig.savefig(OUT / "fig_split.png"); plt.show()
```

4.1.2. Mô hình và huấn luyện

Dữ liệu được chia thành tập huấn luyện (152.208), kiểm định (50.736) và kiểm tra (50.736) (Hình 4.1). Quy trình trong `models/training_pipeline.py`:

1. Năm mô hình ứng viên: hồi quy logistic (đối chứng), rừng ngẫu nhiên, LightGBM, HistGradientBoosting và MLP PyTorch; tất cả dùng cân bằng lớp.
2. CV phân tầng 5 lần trên tập huấn luyện; hai mô hình có CV cao nhất được tuning bằng `RandomizedSearchCV` (12 vòng, `cv=3`).
3. MLP: các lớp ẩn (64, 32) với ReLU và Dropout 0,2; hàm mất mát BCE có `pos_weight`; tối ưu AdamW [14] (lr = 10^{-3}, weight decay 10^{-5}); dừng sớm theo mất mát kiểm định (patience 5). Đây là bộ tham số tốt nhất do tuning chọn.
4. Hiệu chuẩn xác suất bằng hồi quy đẳng điệu (`CalibratedClassifierCV`, `isotonic`, `cv=3`), vì cân bằng lớp làm lệch xác suất.
5. **Chọn mô hình và ngưỡng trên tập kiểm định**; tập kiểm tra chỉ dùng để đánh giá cuối.



Hình 4.2. ROC-AUC của năm mô hình ứng viên (CV, kiểm định, kiểm tra), vẽ từ `metrics.json`

Bảng 4.1. Kết quả năm mô hình (ROC-AUC của CV là trung bình ± độ lệch chuẩn)

Mô hình	CV AUC	Kiểm định	Kiểm tra	AP (kiểm tra)	Brier
Hồi quy logistic	0, 8241 ± 0, 0020	0,8211	0,8197	0,392	0,0993
Rừng ngẫu nhiên	0, 8215 ± 0, 0020	0,8256	0,8240	0,415	0,0980
LightGBM	0, 8284 ± 0, 0024	0,8276	0,8264	0,422	0,0975
HistGradientBoosting	0, 8312 ± 0, 0024	0,8278	0,8266	0,422	0,0974
MLP (được chọn)	0, 8305 ± 0, 0024	0,8283	0,8258	0,414	0,0978

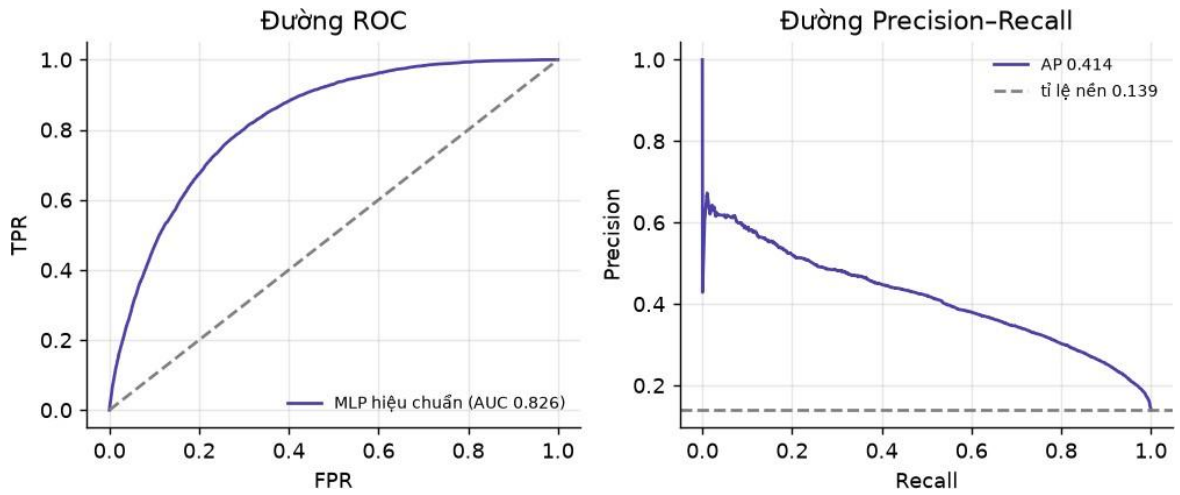
Cell 3 của `training.ipynb`: vẽ Hình 4.2 và ghi Bảng 4.1 ra `tables/tab_models.tex`

```
order = ["logistic_regression", "random_forest", "lightgbm", "hist_gradient_boosting", "mlp"]
labels = {"logistic_regression": "Hồi quy\nlogistic", "random_forest": "Rừng\nngẫu nhiên", "lightgbm":
↪  "LightGBM",
          "hist_gradient_boosting": "HistGradient\nBoosting", "mlp": "MLP\n(PyTorch)"}
M = metrics["metrics"]
cv = [M[m]["cv"]["roc_auc_mean"] for m in order]
cv_sd = [M[m]["cv"]["roc_auc_std"] for m in order]
va = [M[m]["validation"]["roc_auc"] for m in order]
te = [M[m]["test"]["roc_auc"] for m in order]
x = np.arange(len(order)); w = .26
fig, ax = plt.subplots(figsize=(7.2, 3.4))
ax.bar(x - w, cv, w, yerr=cv_sd, capsize=3, label="CV 5-fold (TB ± độ lệch)", color="#c9d3f2")
ax.bar(x, va, w, label="Kiểm định", color="#8a7fd6")
ax.bar(x + w, te, w, label="Kiểm tra", color="#4b3fa8")
ax.set_ylim(.81, .838); ax.set_xticks(x, [labels[m] for m in order]); ax.set_ylabel("ROC-AUC")
ax.legend(fontsize=8, frameon=False, ncol=3, loc="upper center")
fig.tight_layout(); fig.savefig(OUT / "fig_model_compare.png"); plt.show()

# Bảng LaTeX cho báo cáo (Bảng kết quả năm mô hình)
names_vi = {"logistic_regression": "Hồi quy logistic", "random_forest": "Rừng ngẫu nhiên", "lightgbm":
↪  "LightGBM",
            "hist_gradient_boosting": "HistGradientBoosting", "mlp": r"\textbf{MLP (được chọn)}"}
rows = []
for m in order:
```

```
    t = M[m]["test"]
    rows.append(f"{names_vi[m]} & ${vn(M[m]['cv']['roc_auc_mean'])}\\pm{vn(M[m]['cv']['roc_auc_std'])}$ &
    ↪  "
                f"{vn(M[m]['validation']['roc_auc'])} & {vn(t['roc_auc'])} & {vn(t['average_precision'],
                ↪  3)} & {vn(t['brier_score'])} \\\\")
tex = ("\\begin{tabular}{@{}lccccc@{}}\n\\toprule\n"
       "\\textbf{Mô hình} & \\textbf{CV AUC} & \\textbf{Kiểm định} & \\textbf{Kiểm tra} & \\textbf{AP
       ↪  (kiểm tra)} & \\textbf{Brier}\\\\\n\\midrule\n"
       + "\n".join(rows) + "\n\\bottomrule\n\\end{tabular}\n")
(TAB / "tab_models.tex").write_text(tex, encoding="utf-8")
print(pd.DataFrame({"cv": cv, "val": va, "test": te}, index=order).round(5))
```

Nhận xét thẳng thắn. MLP được chọn vì có AUC kiểm định cao nhất (0,8283 so với 0,8278 của HistGradientBoosting), nhưng chênh lệch này nằm trong nhiễu; trên tập kiểm tra HistGradientBoosting thậm chí nhỉnh hơn (0,8266 so với 0,8258). Với 21 đặc trưng dạng bảng, các mô hình boosting và MLP tương đương, và hồi quy logistic chỉ kém khoảng 0,006 AUC. Dự án vẫn dùng MLP để đáp ứng khối "Deep Learning" của sơ đồ, chứ không phải vì học sâu vượt trội.



Hình 4.3. Đường ROC và Precision–Recall của MLP hiệu chuẩn trên tập kiểm tra

Cell 4 của `training.ipynb`: vẽ Hình 4.3

```
from sklearn.metrics import roc_curve, precision_recall_curve, roc_auc_score, average_precision_score
artifact = joblib.load(ROOT / "artifacts" / "risk_model.joblib")
model = artifact["model"]
p_test = model.predict_proba(X_test[artifact["feature_names"]])[:, 1]
print("ROC-AUC", round(roc_auc_score(y_test, p_test), 5), "AP", round(average_precision_score(y_test,
↪  p_test), 5))

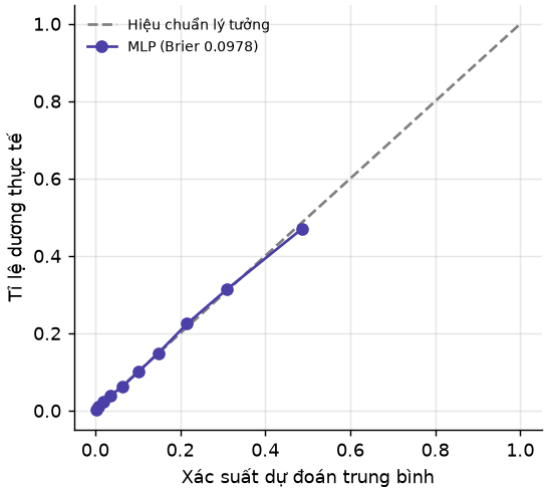
fpr, tpr, _ = roc_curve(y_test, p_test)
prec, rec, _ = precision_recall_curve(y_test, p_test)
fig, axes = plt.subplots(1, 2, figsize=(8, 3.5))
axes[0].plot(fpr, tpr, color="#4b3fa8", label=f"MLP hiệu chuẩn (AUC {roc_auc_score(y_test, p_test):.3f})")
axes[0].plot([0, 1], [0, 1], "--", color="gray"); axes[0].set_xlabel("FPR"); axes[0].set_ylabel("TPR")
axes[0].set_title("Đường ROC"); axes[0].legend(fontsize=8, frameon=False, loc="lower right")
axes[1].plot(rec, prec, color="#4b3fa8", label=f"AP {average_precision_score(y_test, p_test):.3f}")
axes[1].axhline(y_test.mean(), ls="--", color="gray", label=f"ti lệ nền {y_test.mean():.3f}")
axes[1].set_xlabel("Recall"); axes[1].set_ylabel("Precision"); axes[1].set_title("Đường Precision-Recall")
axes[1].legend(fontsize=8, frameon=False)
fig.tight_layout(); fig.savefig(OUT / "fig_roc_pr.png"); plt.show()
```

4.1.3. Hiệu chuẩn, ngưỡng và phân tầng nguy cơ

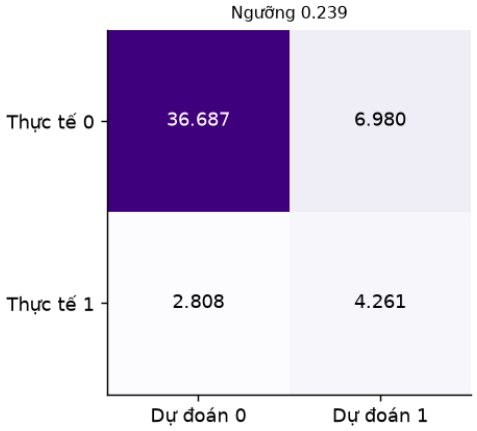
Sau hiệu chuẩn, xác suất dự đoán khớp tỉ lệ dương thực tế (Hình 4.4, Brier 0,0978). Hai khái niệm được tách riêng nhưng cùng chọn trên tập kiểm định:

- **Ngưỡng nhị phân** 0,23906: xác suất cho F1 cao nhất trên kiểm định; ≥ ngưỡng thì gắn cờ dương tính.
- **Ba mức**: Thấp ($p < 0{,}23906$), Trung bình ($0{,}23906 \le p < 0{,}40992$), Cao ($p \ge 0{,}40992$), trong đó mốc Cao là xác suất thấp nhất trên kiểm định mà độ chính xác của các ca được gắn cờ đạt tối thiểu 0,5.

Nhờ vậy mọi ca dương tính đều từ mức Trung bình trở lên; trước đây mốc cố định 0,4/0,7 làm một ca xác suất 0,30 vừa dương tính vừa "Thấp". Trên tập kiểm tra, tỉ lệ nhãn 1 theo mức là 7,1% (Thấp, 39.495 ca), 32,1% (Trung bình, 7.371 ca) và 49,0% (Cao, 3.870 ca), tức phân tầng có ý nghĩa. Tại ngưỡng 0,23906: độ chính xác 0,379, độ nhạy 0,603, F1 0,465 (Hình 4.5).



Hình 4.4. Đường hiệu chuẩn



Hình 4.5. Ma trận nhầm lẫn tại ngưỡng nhị phân

Cell 5 của `training.ipynb`: vẽ Hình 4.4

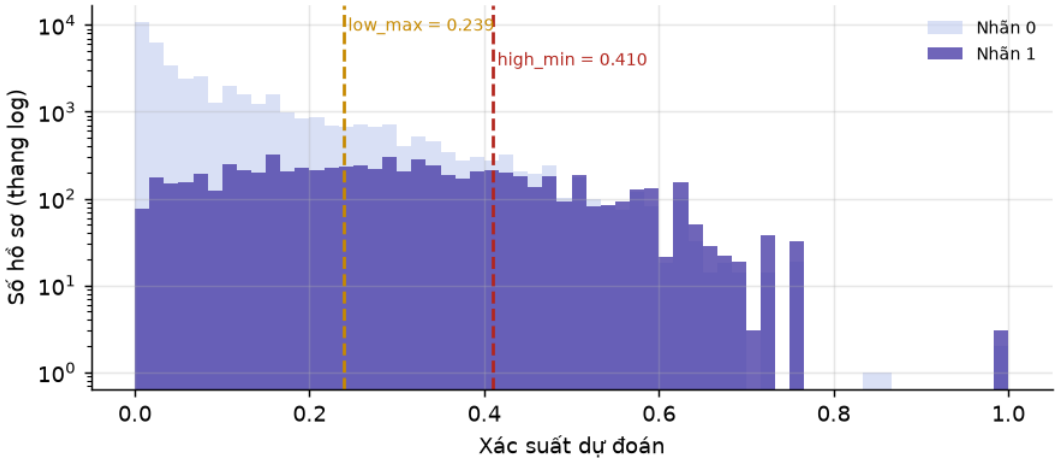
```
from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss
frac, mean_p = calibration_curve(y_test, p_test, n_bins=10, strategy="quantile")
fig, ax = plt.subplots(figsize=(4.4, 4))
ax.plot([0, 1], [0, 1], "--", color="gray", label="Hiệu chuẩn lý tưởng")
ax.plot(mean_p, frac, "o-", color="#4b3fa8", label=f"MLP (Brier {brier_score_loss(y_test, p_test):.4f})")
ax.set_xlabel("Xác suất dự đoán trung bình"); ax.set_ylabel("Tỉ lệ dương thực tế"); ax.legend(fontsize=8,
↪  frameon=False)
fig.tight_layout(); fig.savefig(OUT / "fig_calibration.png"); plt.show()
```

Cell 7 của `training.ipynb`: vẽ Hình 4.5

```
from sklearn.metrics import confusion_matrix
pred = (p_test >= thr).astype(int)
tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
print(dict(tn=tn, fp=fp, fn=fn, tp=tp), "precision", round(tp/(tp+fp), 4), "recall", round(tp/(tp+fn), 4))
```

```
assert (tn, fp, fn, tp) == tuple(metrics["metrics"]["mlp"]["test"]["confusion_matrix"].values())

fig, ax = plt.subplots(figsize=(3.8, 3.4))
cm = np.array([[tn, fp], [fn, tp]])
ax.imshow(cm, cmap="Purples"); ax.grid(False)
for i in range(2):
    for j in range(2):
        ax.text(j, i, f"{cm[i, j]:,}".replace(",", "."), ha="center", va="center", color="white" if cm[i,
        ↪  j] > 20000 else "black")
ax.set_xticks([0, 1], ["Dự đoán 0", "Dự đoán 1"]); ax.set_yticks([0, 1], ["Thực tế 0", "Thực tế 1"])
ax.set_title(f"Ngưỡng {thr:.3f}", fontsize=9)
fig.tight_layout(); fig.savefig(OUT / "fig_confusion.png"); plt.show()
```



Hình 4.6. Phân bố xác suất theo nhãn và hai mốc phân tầng (thang log)

Bảng 4.2. Tỉ lệ nhãn 1 trên tập kiểm tra theo mức nguy cơ

Mức	Số ca	Nhãn 1	Tỉ lệ nhãn 1
Thấp	39.495	2.808	7,1%
Trung bình	7.371	2.365	32,1%
Cao	3.870	1.896	49,0%

Cell 6 của `training.ipynb`: vẽ Hình 4.6 và ghi Bảng 4.2

```
bands = metrics["risk_bands"]; thr = metrics["selected_threshold"]
fig, ax = plt.subplots(figsize=(7, 3.2))
bins = np.linspace(0, 1, 61)
ax.hist(p_test[y_test == 0], bins=bins, alpha=.7, color="#c9d3f2", label="Nhãn 0")
ax.hist(p_test[y_test == 1], bins=bins, alpha=.8, color="#4b3fa8", label="Nhãn 1")
ax.set_yscale("log")
ax.axvline(bands["low_max"], color="#c78a00", ls="--"); ax.axvline(bands["high_min"], color="#b3261e",
↪  ls="--")
ymax = ax.get_ylim()[1]
ax.text(bands["low_max"] + .005, ymax * .5, f"low_max = {bands['low_max']:.3f}", color="#c78a00",
↪  fontsize=8)
ax.text(bands["high_min"] + .005, ymax * .2, f"high_min = {bands['high_min']:.3f}", color="#b3261e",
↪  fontsize=8)
```

```
ax.set_xlabel("Xác suất dự đoán"); ax.set_ylabel("Số hồ sơ (thang log)"); ax.legend(fontsize=8,
↪  frameon=False)
fig.tight_layout(); fig.savefig(OUT / "fig_bands.png"); plt.show()

from tools.risk_classifier import classify_risk
level = pd.Series([classify_risk(p, bands) for p in p_test])
rows = []
for lv, name in [("Low", "Thấp"), ("Medium", "Trung bình"), ("High", "Cao")]:
    mask = (level == lv).values
    n, k = int(mask.sum()), int(y_test.values[mask].sum())
    dot = lambda v: f"{v:,}".replace(",", ".")          # dấu chấm ngăn cách hàng nghìn
    rows.append(f"{name} & {dot(n)} & {dot(k)} & {vn(100 * k / n, 1)}\\% \\\\")
tex = ("\\begin{tabular}{@{}lccc@{}}\n\\toprule\n\\textbf{Mức} & \\textbf{Số ca} & \\textbf{Nhãn 1} &
↪  \\textbf{Tỉ lệ nhãn 1}\\\\\n\\midrule\n"
       + "\n".join(rows) + "\n\\bottomrule\n\\end{tabular}\n")
(TAB / "tab_bands.tex").write_text(tex, encoding="utf-8")
print(level.value_counts())
```

4.1.4. Từ mô hình thành thành phần của hệ thống

Một con số xác suất chưa đủ để tư vấn. Quanh mô hình, `RiskModel` và `FeatureExplainer` bổ sung:

1. **Chuẩn hóa đầu vào**: người dùng chỉ cần tuổi, huyết áp tâm thu và BMI (hoặc chiều cao, cân nặng); tuổi đổi thành nhóm BRFSS; `HighBP` suy ra từ huyết áp ≥ 130 mmHg nếu người dùng không khai (xấp xỉ, ghi rõ là hạn chế); các mục tùy chọn thiếu được điền giá trị mặc định và **được đánh dấu** (`defaulted_fields`).
2. **Giải thích**: phân tích độ nhạy, tức thay từng đặc trưng bằng giá trị mặc định (trung vị huấn luyện) rồi đo xác suất đổi bao nhiêu; trường tự điền bị đánh dấu để không giải thích như thể người dùng đã khai.
3. **Điểm chất lượng dữ liệu** (`confidence_score`): điểm quy ước trừ theo tỉ lệ mục tùy chọn bị tự điền (tối đa 0,2), huyết áp ≤ 70 hoặc ≥ 220 mmHg ($-0{,}15$) và BMI ≤ 14 hoặc ≥ 50 ($-0{,}1$). *Đây không phải độ tin cậy thống kê*, và giao diện ghi rõ như vậy.
4. **Hồ sơ tương tự**: tìm theo khoảng cách số trên tuổi, BMI, huyết áp và glucose (khi cả hai đều đo); mỗi hồ sơ mang `outcome_confirmed=false`, tức là dự đoán cũ chứ không phải kết quả bệnh đã xác nhận.

4.2. Knowledge Base và RAG

4.2.1. Nguồn tri thức và đồ thị Neo4j

Tri thức được tuyển chọn thủ công trong `data/kb/diabetes_kb.json` (phiên bản 9) từ năm nguồn: ADA Standards of Care 2025 [1], WHO [3], CDC [4], NIDDK [5] và thử nghiệm DPP [2]. Nội dung bằng chứng là diễn giải ngắn từ các tài liệu này; **chưa được đối chiếu lại từng URL tự động và chưa được chuyên gia y khoa thẩm định**. Script `scripts/load_neo4j_kb.py` kiểm tra tính nhất quán của tệp (nguồn, thực thể, quan hệ) rồi nạp vào Neo4j bằng `MERGE`, nên chạy lại an toàn; mục không còn trong tệp bị xoá trừ khi dùng `--no-prune`.

Lược đồ gồm bốn nhãn thực thể (`Condition`, `RiskFactor`, `Test`, `Intervention`), nút `Evidence` và `Source`, và năm loại quan hệ: `INCREASES_RISK_OF`, `DIAGNOSES`, `REDUCES_RISK_OF`, `ADDRESSES`, `SHOULD_TEST_WITH`. Mỗi quan hệ mang `evidence_ids`; mỗi `Evidence` nối tới `Source` qua `FROM`, nên mọi khẳng định truy vấn ra đều đi kèm trích dẫn. Kết quả: 20 thực thể, 35 quan hệ nghiệp vụ, 21 đoạn bằng chứng, 5 nguồn; sau khi nạp, Neo4j có 46 nút và 56 quan hệ (gồm 21 quan hệ bằng chứng → nguồn), được xác minh ngày 03/11/2026 trên Neo4j 2026.10.1 chạy trong Docker.

Hình 4.7 vẽ đồ thị tri thức từ chính tệp `data/kb/diabetes_kb.json` (tệp được nạp vào Neo4j) bằng NetworkX; hình chỉ gồm 20 nút thực thể và 35 quan hệ, còn các nút `Evidence` và `Source` không được vẽ cho gọn. Bảng 4.3 thống kê số lượng, cùng cell sinh ra chúng.



Hình 4.7. Đồ thị tri thức về ĐTĐ (nút tô màu theo nhãn: Condition, RiskFactor, Test, Intervention)

Bảng 4.3. Thống kê nút và quan hệ của KB

Nút / dữ liệu	SL	Loại quan hệ	SL
Condition	3	ADDRESSES	8
Intervention	7	DIAGNOSES	6
RiskFactor	7	INCREASES_RISK_OF	11
Test	3	REDUCES_RISK_OF	6
Đoạn bằng chứng	21	SHOULD_TEST_WITH	4
Nguồn	5		

Cell 8 của `training.ipynb`: vẽ Hình 4.7 và ghi Bảng 4.3

```
import networkx as nx
from collections import Counter
kb = json.loads((ROOT / "data" / "kb" / "diabetes_kb.json").read_text(encoding="utf-8-sig"))
G = nx.DiGraph()
for e in kb["entities"]:
    G.add_node(e["id"], name=e["name"], label=e["label"])
for r in kb["relations"]:
    G.add_edge(r["from"], r["to"], type=r["type"])
colors = {"Condition": "#d64545", "RiskFactor": "#f0a030", "Test": "#3b73d9", "Intervention": "#2f9e6b"}
pos = nx.spring_layout(G.to_undirected(), seed=7, k=1.6, iterations=400)
import textwrap
fig, ax = plt.subplots(figsize=(8.6, 6.8)); ax.axis("off")
nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#9aa0b4", arrows=True, arrowsize=8, width=.8, alpha=.8,
                       connectionstyle="arc3,rad=0.06", node_size=2300)
nx.draw_networkx_nodes(G, pos, ax=ax, node_size=2300, node_color=[colors[G.nodes[n]["label"]] for n in G],
↪  alpha=.95)
nx.draw_networkx_labels(G, pos, labels={n: textwrap.fill(G.nodes[n]["name"], 14) for n in G},
                        font_size=5.2, ax=ax)
for lab, c in colors.items():
    ax.scatter([], [], c=c, label=lab, s=60)
ax.legend(loc="upper center", bbox_to_anchor=(.5, 0.02), ncol=4, fontsize=8, frameon=False)
fig.tight_layout(); fig.savefig(OUT / "fig_kg.png", dpi=170); plt.show()

# Bảng thống kê KB cho báo cáo
ent = Counter(e["label"] for e in kb["entities"]); rel = Counter(r["type"] for r in kb["relations"])
left = [f"{k} & {v}" for k, v in sorted(ent.items())] + ["Đoạn bằng chứng & " + str(len(kb["evidence"])),
↪  "Nguồn & " + str(len(kb["sources"]))]
right = [f"{k.replace('_', chr(92) + '_')} & {v}" for k, v in sorted(rel.items())]
n = max(len(left), len(right)); left += ["&"] * (n - len(left)); right += ["&"] * (n - len(right))
body = "\n".join(f"{a} & {b} \\\\" for a, b in zip(left, right))
tex = ("\\begin{tabular}{@{}lr@{\\hspace{2em}}lr@{}}\n\\toprule\n\\textbf{Nút / dữ liệu} & \\textbf{SL} &
↪  \\textbf{Loại quan hệ} & \\textbf{SL}\\\\\n\\midrule\n"
       + body + "\n\\bottomrule\n\\end{tabular}\n")
(TAB / "tab_kb.tex").write_text(tex, encoding="utf-8")
print(len(kb["entities"]), "thực thể,", len(kb["relations"]), "quan hệ,", len(kb["evidence"]), "bằng
↪  chứng,", len(kb["sources"]), "nguồn")
```

4.2.2. Truy xuất: ngữ nghĩa kết hợp đồ thị

`RetrievalAgent` thực hiện [12]:

1. Dựng câu truy vấn từ mức nguy cơ, các yếu tố nguy cơ có tác động dương trong giải thích, và việc glucose đã đo hay chưa.

2. Nhúng bằng `all-MiniLM-L6-v2` và tính độ tương đồng cosine với các đoạn bằng chứng trong ChromaDB (chỉ mục HNSW).
3. Loại đoạn có độ liên quan dưới **0,3**; đoạn mà đồ thị gắn với yếu tố nguy cơ của bệnh nhân được cộng **0,15** khi xếp hạng.
4. Gán `evidence_status`: `none` (không có đoạn nào), `limited` (dưới 2 đoạn hoặc đoạn tốt nhất dưới 0,45), `sufficient` (còn lại).

Khi policy chọn "tìm thêm bằng chứng", agent chạy thêm một truy vấn hướng dẫn chung, giữ điểm tốt nhất của mỗi đoạn và trả tối đa 5 đoạn. Nếu Neo4j không kết nối được lúc khởi tạo (hoặc chưa đặt `NEO4J_PASSWORD`), hệ thống dùng `FileKnowledgeBase` đọc cùng tệp JSON, và `kb_source` cho biết đang dùng nguồn nào.

Listing 4.1. Rút gọn RetrievalAgent.retrieve (agents/retriever.py)

```
factors = self._risk_factors(patient_data, feature_explanation)
queries = [self._query_text(patient_data, risk, factors)]
if broaden:                        # policy asked for more evidence
    queries.append(BROAD_QUERY)
    n_results = max(n_results, BROAD_N_RESULTS)
for query in queries:              # keep each passage's best cosine score
    res  =  self.collection.query(query_embeddings=[embed(query)],  n_results=count)
    ...
linked = self._graph_links(factors)          # evidence id -> graph rows (Neo4j)
for evidence_id, score in relevance.items():
    if score < self.min_relevance:           # 0.3
        continue
    ...  # "graph+semantic" if linked else "semantic"
contexts.sort(key=lambda c: c["relevance"] + (GRAPH BOOST if c["supports"] else 0))
```

Giới hạn đánh giá. Dự án **chưa** có bộ câu hỏi đánh giá truy xuất có đáp án (không đo được hit@*k* hay MRR). Các ngưỡng 0,3, 0,45 và 0,15 được đặt theo một số ca thử, chưa hiệu chỉnh trên tập đánh giá; kiểm thử hiện tại (`tests/test_retriever.py`) chỉ kiểm tra hành vi (không bằng chứng, bằng chứng hạn chế, thứ tự, dự phòng). Embedding MiniLM chủ yếu dành cho tiếng Anh, còn KB lưu bằng chứng tiếng Anh và có bản diễn đạt tiếng Việt để hiển thị.

4.3. Bộ luật và Reinforcement Learning

4.3.1. Bộ luật an toàn và xử lý xung đột

Bộ luật (`tools/rules.py`) chạy hai pha: luật *cơ sở* xét số đo, dự đoán và chất lượng dữ liệu; luật *xung đột* xét sự bất đồng giữa các kết quả đó và bằng chứng. Mỗi luật có mã, tên, nhóm, mức ưu tiên (số lớn hơn quan trọng hơn), điều kiện, thông điệp, mức chuyển tuyến và căn cứ. Các ngưỡng là *ngưỡng của dự án*, có đối chiếu với ADA và khuyến cáo ACC/AHA [6], **chưa được bác sĩ thẩm định**.

Bảng 4.4. Các luật hiện có trong `tools/rules.py`

Mã luật	Ưu tiên	Điều kiện	Hướng xử trí
`GLU-300`	100	Glucose đo được ≥ 300 mg/dL	Khám khẩn cấp
`BP-180`	95	Huyết áp tâm thu ≥ 180 mmHg	Khám khẩn cấp
`GLU-200`	80	Glucose đo được 200–299	Sớm khám bác sĩ
`BP-160`	75	Huyết áp tâm thu 160–179	Sớm khám bác sĩ
`PRED-VERYHIGH`	60	Mức Cao và $p \geq 0{,}85$	Sớm khám bác sĩ
`CF-PRED-LOW-MEASURED-HIGH`	85	Mô hình Thấp nhưng có số đo nguy hiểm	Số đo thắng; `effective_risk`=Cao
`CF-PRED-HIGH-GLUCOSE-NORMAL`	55	Mô hình Cao nhưng glucose đo < 100	Giữ mức Cao, đề nghị A1C
`CF-PRED-HIGH-LOW-QUALITY`	40	Mô hình Cao trên dữ liệu chất lượng thấp	Đánh dấu tạm thời
`CF-ESCALATION-NO-EVIDENCE`	30	Có chuyển tuyến nhưng bằng chứng `none`/`limited`	Không hạ mức chuyển tuyến

Chính sách giải quyết: (1) số đo lâm sàng thắng dự đoán thống kê, khuyến nghị viết theo `effective_risk`; (2) mức chuyển tuyến là mức nặng nhất trong các luật đã kích hoạt, không gì hạ được, kể cả dự đoán thấp hay thiếu bằng chứng; (3) mọi xung đột đều được báo cáo kèm cách giải quyết, không bị ẩn. Luật glucose chỉ áp dụng cho giá trị *đo thật*, không dùng giá trị điền thay thế. Việc chưa đo glucose **không** còn là luật cảnh báo riêng: thay vào đó giao diện hỏi bổ sung (Mục 4.4).

4.3.2. Agent chọn bước hỗ trợ bằng Q-learning

Policy (`rl/`) mô hình hóa việc chọn bước tiếp theo như một MDP; Q-learning [15, 16] cập nhật $Q(s, a) \leftarrow Q(s, a) + \alpha\,[\, r + \gamma \max_{a'} Q(s', a') - Q(s, a)\,]$ với $\alpha = 0{,}1$ và $\gamma = 1$ (bài toán có kết thúc hữu hạn).

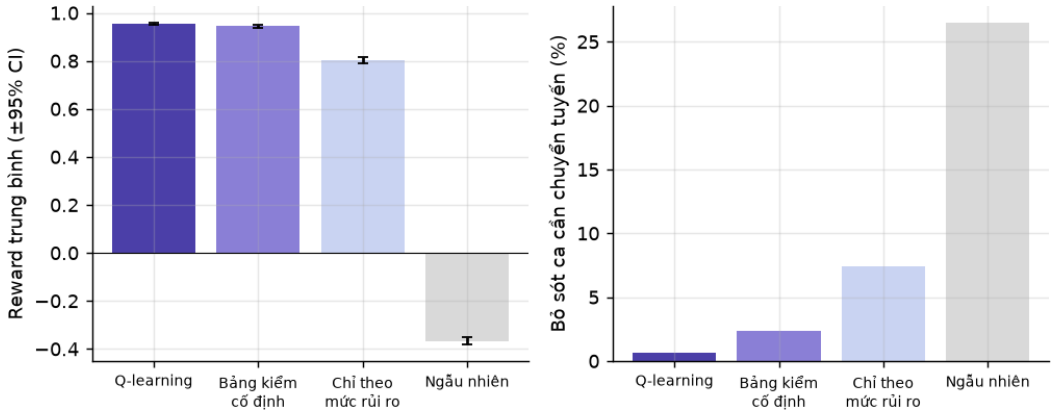
- **Trạng thái** ($108 = 3 \times 3 \times 3 \times 2 \times 2$): mức nguy cơ (Thấp/Trung bình/Cao); glucose (chưa đo/bình thường/≥ 200); `evidence_status`; đã hỏi glucose chưa; đã tìm mở rộng chưa.
- **Hành động** (5): `request_glucose`, `retrieve_more_evidence` (mỗi bước tối đa một lần), và ba bước kết thúc `recommend_routine`, `recommend_followup`, `escalate_to_clinician`.
- **Phần thưởng**: mỗi bước thu thập $-0{,}05$; bước kết thúc $+1$ nếu đúng mức cần, phạt khi sai, nặng nhất khi bỏ sót ca cần chuyển tuyến (-2), nhẹ khi chuyển tuyến thừa ($-0{,}4$ đến $-0{,}8$); lời khuyên không chuyển tuyến cộng $+0{,}2$ nếu bằng chứng đủ, trừ $0{,}2$ nếu không có.
- **Huấn luyện**: 60.000 tập, ϵ giảm tuyến tính từ 1 xuống 0,05 trong 80% số tập đầu.

Hạn chế cần nói rõ. Môi trường, xác suất bệnh nhân cung cấp glucose, khả năng tìm mở rộng

cải thiện bằng chứng (giả định 60%) và mọi giá trị phần thưởng đều do nhóm đặt, ghi trong `rl/environment.py`; **không có phản hồi thật** từ bệnh nhân hay bác sĩ. Kết quả chỉ cho biết các chiến lược khác nhau ra sao *dưới các giả định đó*.

Bảng 4.5. So sánh chiến lược trên 20.000 tập mô phỏng

Chiến lược	Reward TB ± 95% CI	Số bước	Bỏ sót	Chuyển thừa
Q-learning	$0,956 \pm 0,005$	1,78	0,65%	3,55%
Bảng kiểm cố định	$0,946 \pm 0,006$	2,10	2,34%	0,00%
Chỉ theo mức rủi ro	$0,803 \pm 0,011$	1,00	7,42%	0,00%
Ngẫu nhiên	$-0,367 \pm 0,016$	1,28	26,47%	20,09%



Hình 4.8. Reward trung bình và tỉ lệ bỏ sót ca cần chuyển tuyến của bốn chiến lược (mô phỏng)

Cell 9 của `training.ipynb`: vẽ Hình 4.8 và ghi Bảng 4.5

```
comp = json.loads((ROOT / "artifacts" / "policy_comparison.json").read_text(encoding="utf-8"))
keys = ["q_learning", "fixed_full_checklist", "fixed_risk_only", "random"]
names = ["Q-learning", "Bảng kiểm\ncố định", "Chỉ theo\nmức rủi ro", "Ngẫu nhiên"]
rew = [comp[k]["mean_reward"] for k in keys]; ci = [comp[k]["ci95"] for k in keys]
under = [comp[k]["under_triage_rate"] * 100 for k in keys]
fig, axes = plt.subplots(1, 2, figsize=(8, 3.3))
axes[0].bar(names, rew, yerr=ci, capsize=3, color=["#4b3fa8", "#8a7fd6", "#c9d3f2", "#d9d9d9"])
axes[0].set_ylabel("Reward trung bình (±95% CI)"); axes[0].axhline(0, color="black", lw=.6)
axes[1].bar(names, under, color=["#4b3fa8", "#8a7fd6", "#c9d3f2", "#d9d9d9"])
axes[1].set_ylabel("Bỏ sót ca cần chuyển tuyến (%)")
for a in axes: a.tick_params(axis="x", labelsize=7.5)
fig.tight_layout(); fig.savefig(OUT / "fig_rl_compare.png"); plt.show()

label = ["Q-learning", "Bảng kiểm cố định", "Chỉ theo mức rủi ro", "Ngẫu nhiên"]
rows = [f"{l} & ${vn(comp[k]['mean_reward'], 3)}\\pm{vn(comp[k]['ci95'], 3)}$ & {vn(comp[k]['mean_steps'],
↪  2)} & "
        f"{vn(comp[k]['under_triage_rate'] * 100, 2)}\\% & {vn(comp[k]['over_escalation_rate'] * 100,
        ↪  2)}\\% \\\\"
        for l, k in zip(label, keys)]
tex = ("\\begin{tabular}{@{}lcccc@{}}\n\\toprule\n\\textbf{Chiến lược} & \\textbf{Reward TB $\\pm$ 95\\%
↪  CI} & \\textbf{Số bước} & \\textbf{Bỏ sót} & \\textbf{Chuyển thừa}\\\\\n\\midrule\n"
       + "\n".join(rows) + "\n\\bottomrule\n\\end{tabular}\n")
(TAB / "tab_rl.tex").write_text(tex, encoding="utf-8")
print(comp["q_learning"])
```

Q-learning và bảng kiểm cố định chênh nhau trong khoảng sai số, nên **không thể nói RL tốt hơn bảng kiểm**; cả hai vượt rõ quy trình chỉ theo mức rủi ro. Policy học được dùng ít bước hơn và bỏ sót ít hơn, đổi lại chuyển tuyến thừa nhiều hơn, và kết quả phụ thuộc vào ma trận phần thưởng.

4.3.3. Luật ràng buộc chính sách học được

Orchestrator dựng trạng thái sau lượt xử lý đầu và hỏi policy tối đa 3 bước. Chỉ "tìm thêm bằng chứng" được thực thi thật (chạy lại truy xuất mở rộng, luật và khuyến nghị); "hỏi glucose" không thực hiện được trong một lời gọi API nên chỉ trả về làm `policy.next_step`. **Luật luôn thắng**: nếu policy chọn bước thấp hơn mức luật đã yêu cầu, `next_step` bị thay bằng `escalate_to_clinician` và `overridden_by_rules=true`; policy không bao giờ thay đổi kết quả `safety`.

4.4. Tư vấn, trợ giúp và cô lập lỗi

Khuyến nghị có nguồn. `RecommendationAgent` chỉ dùng các đoạn bằng chứng đã đánh số, trích dẫn `[n]` kèm danh sách nguồn, và nói rõ khi thiếu bằng chứng thay vì bịa nguồn. Với `effective_risk` nâng lên do số đo, khuyến nghị viết theo mức đã nâng.

Thang chỉ số và câu hỏi bổ sung. API trả `measurement_scales` (BMI, huyết áp tâm thu, glucose) cùng vùng tham khảo có nguồn (CDC, AHA, NIDDK). Thang glucose chỉ có vùng tham khảo khi người dùng xác nhận đo lúc đói; huyết áp chỉ có số tâm thu nên không kết luận "bình thường"; BMI chỉ phân vùng người lớn từ 20 tuổi. Hệ thống trả tối đa một `followup`: trước hết là glucose còn thiếu, sau đó là mục chưa khai mà câu trả lời làm xác suất MLP đổi nhiều nhất (chênh tối thiểu 2 điểm phần trăm). Mục đã trả lời hoặc đã bấm "Bỏ qua" không bị hỏi lại, và khi luật yêu cầu bác sĩ thì không đưa nhiệm vụ nhập liệu lên trước.

Mô phỏng "nếu thay đổi" (`WhatIfSimulator`) tính lại xác suất khi giảm 5% cân nặng (nếu BMI $\geq$ 25), vận động đều, bỏ thuốc lá, hoặc đưa huyết áp tâm thu xuống dưới 130 mmHg, chỉ với yếu tố mà người dùng *thực sự khai* và có thể thay đổi, và gắn bằng chứng cho từng kịch bản. Chỉ chạy khi mức chuyển tuyến là theo dõi định kỳ.

Trao đổi giữa các thành phần. Khi các kết luận bất đồng, hệ thống ghi lại một đoạn hội thoại có kiểu (`agents/dialogue.py`, trường `trace.dialogue`). Mỗi thông điệp có người gửi, người nhận, performative, nội dung và vị trí trong luồng (`after_step`: số bước đã chạy khi thông điệp được gửi), nên giao diện xen hội thoại đúng chỗ trong dòng thời gian các bước thay vì tách thành một khối riêng; `SafetyGuard` gửi `challenge` tới thành phần mà một luật xung đột (hoặc việc ghi đè policy) làm mất hiệu lực, và người nhận trả lời bằng hàm phản hồi của chính nó (Bảng 4.6). Mô hình *chấp nhận* khi số đo thực tế thắng, nhưng *giữ nguyên và đề xuất* xét nghiệm A1C khi glucose bình thường mà dự đoán vẫn cao, vì glucose không phải đặc trưng của mô hình. `RetrievalAgent` *đề xuất* tìm mở rộng bằng chứng; đây là đề xuất duy nhất mà Orchestrator **thực hiện thật**: nó chạy lại truy xuất mở rộng, bộ luật và khuyến nghị (tối đa một lần, trước khi policy chạy), rồi `RetrievalAgent` báo kết quả bằng `inform` (số đoạn bằng chứng,

trạng thái bằng chứng, hoặc lỗi nếu tìm thất bại và giữ nguyên các đoạn cũ).

Bảng 4.6. Các cuộc trao đổi giữa thành phần (`agents/dialogue.py`)

Nguyên nhân	Người bị thách thức	Phản hồi
`CF-PRED-LOW-MEASURED-HIGH`	`RiskModel`	`accept`: nâng nguy cơ hiệu lực lên Cao theo số đo
`CF-PRED-HIGH-GLUCOSE-NORMAL`	`RiskModel`	`propose`: giữ mức Cao, đề nghị xét nghiệm A1C
`CF-PRED-HIGH-LOW-QUALITY`	`RiskModel`	`accept`: đánh dấu dự đoán là tạm thời
`CF-ESCALATION-NO-EVIDENCE`	`RetrievalAgent`	`propose` tìm mở rộng (**được thực hiện**); sau đó `inform` kết quả
Luật ghi đè bước của policy	`QPolicy`	`accept`: chấp nhận bị ghi đè, chuyển bác sĩ

Cần nói thẳng về phạm vi của cơ chế này:

- Các hàm phản hồi là *tất định* và được viết sẵn, không có LLM và không có nhiều vòng thương lượng.
- Chỉ một loại thông điệp **có tác động**: đề xuất tìm mở rộng. Nó đổi tập bằng chứng, trạng thái bằng chứng, khuyến nghị và trích dẫn, và có thể làm luật `CF-ESCALATION-NO-EVIDENCE` hết kích hoạt. Nó *không* thể hạ mức chuyển tuyến, vì mức này do các luật cơ sở quyết định và các luật cơ sở không đọc bằng chứng.
- Các thông điệp còn lại (phản hồi của `RiskModel` và `QPolicy`) chỉ **ghi lại lý do luật thắng**: mức nguy cơ hiệu lực và việc ghi đè policy do bộ luật quyết định, không phải do cuộc trao đổi.
- Với KB hiện tại (21 đoạn bằng chứng), các hồ sơ thử đều có bằng chứng đủ, nên đề xuất tìm mở rộng **chưa kích hoạt trên dữ liệu thật**; hành vi này mới được kiểm chứng bằng kiểm thử với retriever giả cho kết quả yếu ở lần tìm đầu (`tests/test_dialogue.py`).

Đây là bản tối thiểu của giao tiếp agent–agent, đủ để truy vết "ai phản biện ai, vì sao, và việc gì đã được làm theo", chưa phải một giao thức thương lượng đầy đủ.

Cô lập lỗi. Mỗi bước tùy chọn chạy trong `RunLog.guard`; lỗi được ghi vào `degraded`, bước đó dùng nội dung dự phòng, và lời cảnh báo "kết quả chưa đầy đủ" được thêm vào `disclaimers`. Riêng bộ luật: nếu lỗi, kết quả nghiêng về `prompt_clinician_followup` với độ tin cậy Thấp, *không* coi là an toàn. Lỗi mô hình trả HTTP 500, không tạo xác suất giả.

CHƯƠNG 5. CODE VÀ DEMO

5.1. Cấu trúc mã nguồn

Listing 5.1. Cấu trúc thư mục chính của dự án

```
healthcare-agent-system/
|-- main.py              build_agent(): nap model, KB, memory, policy
|-- api/app.py           FastAPI: /v1/predict, /history, /health, xac thuc Bearer
|-- agents/              orchestrator, dialogue, retriever, explainer, recommender
|-- models/              risk_model.py, mlp.py (PyTorch->NumPy), training_pipeline.
    py
|-- rl/                  environment.py (mo phong), policy.py (Q-learning)
|-- tools/               rules.py, safety.py, neo4j_kb.py, explainability.py,
|                        what_if.py, patient_guidance.py, audit_logger.py
|-- memory/store.py      MemoryAgent (ChromaDB)
|-- data/                BRFSS 2015 CSV, kb/diabetes_kb.json
|-- artifacts/           risk_model.joblib, metrics.json, support_policy.json
|-- scripts/             download_training_data, train_diabetes_model,
|                        train_support_policy, load_neo4j_kb, demo_step12
|-- web/                 Next.js: app/, components/risk-console/, lib/
|-- ui/app.py            ban Streamlit
|-- tests/               154 kiem thu tu dong
|-- docs/, infra/aws/    tai lieu, Terraform App Runner
```

5.2. Các hàm chính

Bảng 5.1. Các hàm và lớp chính

Hàm / lớp	Vai trò
`Orchestrator.run`	Điều phối một lượt đánh giá, ghi `trace`, trả kết quả có cấu trúc
`Orchestrator._choose_next_step`	Vòng policy tối đa 3 bước; áp quyền ưu tiên của luật
`Orchestrator._negotiate`	Gửi `challenge` theo luật xung đột, thực hiện đề xuất tìm mở rộng của retriever (một lần), ghi kết quả bằng `inform`
`conflict_exchanges`, `override_exchange`	Dựng các thông điệp `challenge`/`accept`/`propose`/`inform` giữa các thành phần bất đồng
`RunLog.guard`	Bọc một bước tùy chọn: lỗi thành `degraded` và nội dung dự phòng
`RiskModel.assess`	Chuẩn hóa hồ sơ, ghi trường tự điền, trả xác suất đã hiệu chuẩn
`train_production_model`	CV, tuning, hiệu chuẩn, chọn mô hình và ngưỡng trên kiểm định
`TorchMLPClassifier`	MLP huấn luyện bằng PyTorch, suy luận bằng NumPy
`FeatureExplainer.explain`	Phân tích độ nhạy theo từng đặc trưng
`RetrievalAgent.retrieve`	Truy xuất ngữ nghĩa + liên kết đồ thị, tính `relevance`
`KnowledgeGraph`	Truy vấn Neo4j: `support_for_risk_factors`, `all_evidence`
`RuleEngine.evaluate`	Luật cơ sở rồi luật xung đột; trả `escalation`, `effective_risk`
`SafetyGuard.assess`	Chạy bộ luật và tính điểm chất lượng dữ liệu
`QPolicy.decide`	Chọn hành động theo bảng Q cho trạng thái hợp lệ
`WhatIfSimulator.scenarios`	Tính lại xác suất cho các kịch bản thay đổi
`build_followup`	Chọn một câu hỏi bổ sung theo thứ tự ưu tiên

Listing 5.2. Orchestrator: bước tùy chọn không làm hỏng cả yêu cầu (rút gọn)

```
retrieved = log.guard("retriever",
    lambda: self.retriever.retrieve(patient, risk, feature_explanation, broaden=
    broaden),
    list(previous),                        # fallback: keep earlier passages
    detail=lambda v: f"{len(v)} evidence passages",
    on_failure="keep previous passages")
# a failed rule check must not read as "all clear": fail toward clinician review
safety = log.guard("safety_guard",
    lambda: self.safety_guard.assess(patient, prob, risk, evidence_status=status),
    {"escalation": "prompt_clinician_followup", "confidence_label": "Low", ...})
```

5.3. Cách chạy

Listing 5.3. Cài đặt và chạy hệ thống (PowerShell)

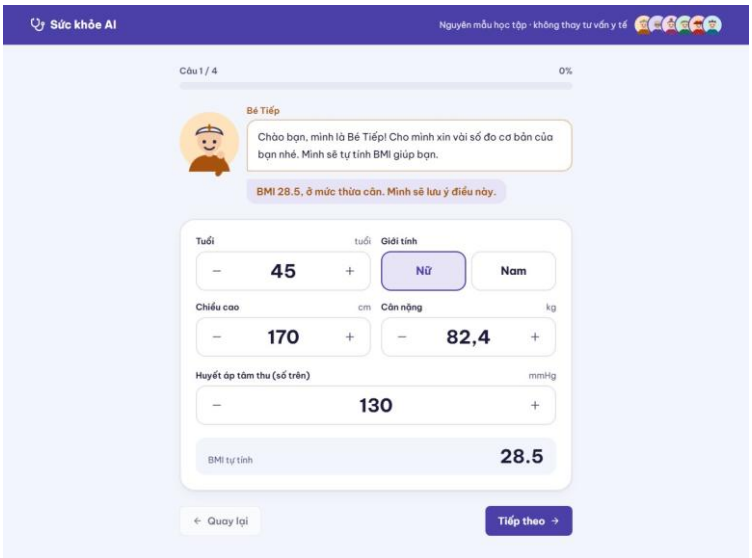
```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe  scripts/download_training_data.py   # BRFSS 2015
.\.venv\Scripts\python.exe  scripts/train_diabetes_model.py      # artifacts/
.\.venv\Scripts\python.exe scripts/train_support_policy.py     # policy Q-learning
docker compose up -d neo4j                                       # tuy chon
.\.venv\Scripts\python.exe  scripts/load_neo4j_kb.py             # nap KB vao Neo4j
.\.venv\Scripts\python.exe -m uvicorn api.app:app --reload --env-file .env
cd web; npm ci; npm run dev                                      # http://localhost
    :3000
.\.venv\Scripts\python.exe -m pytest -q                          # kiem thu
```

Biến môi trường chính: `NEO4J_PASSWORD` (đặt thì dùng Neo4j, không thì đọc KB từ tệp), `OPENAI_API_KEY` (tùy chọn), `REQUIRE_API_TOKEN` và `API_TOKEN` (xác thực Bearer).

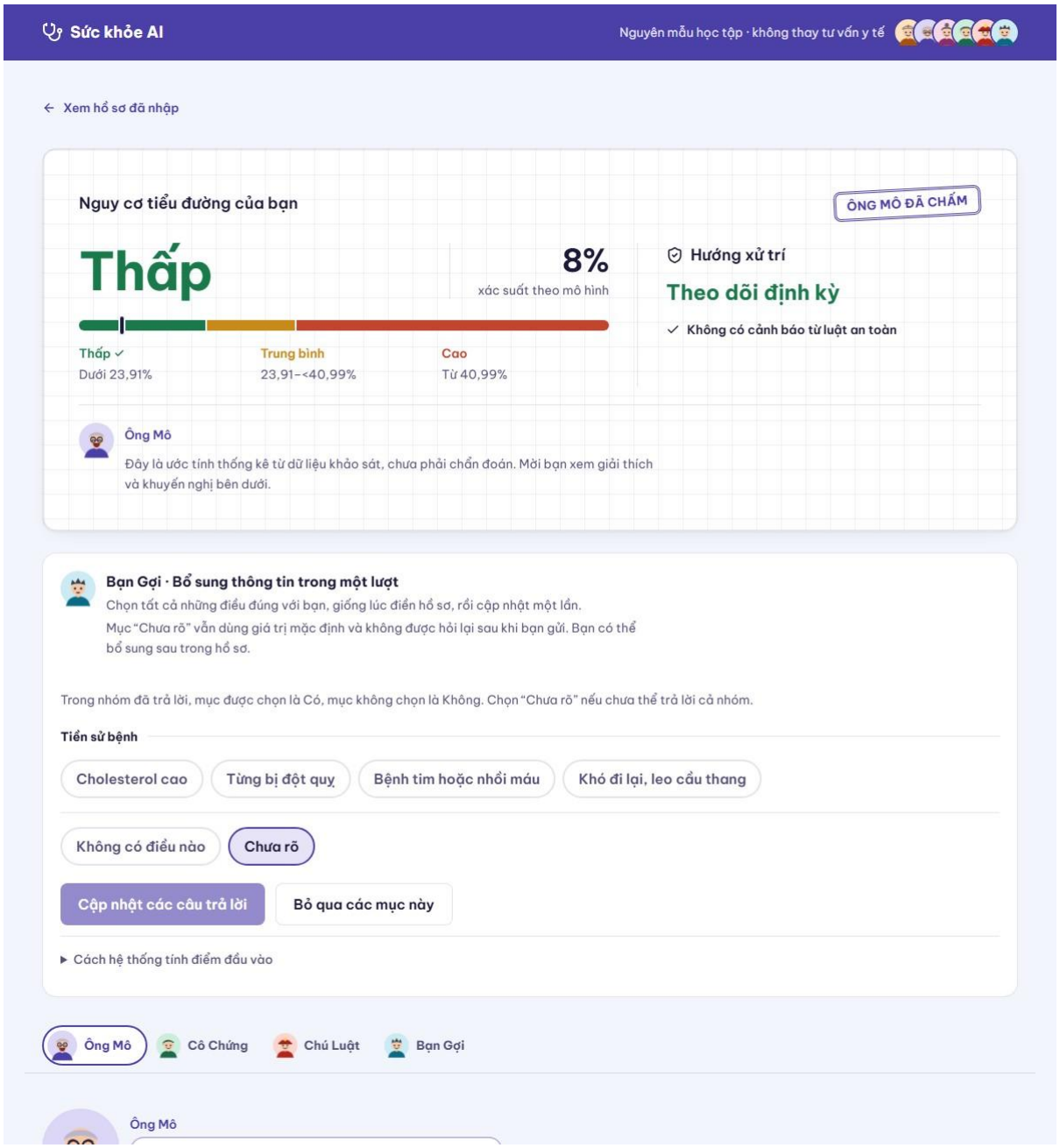
5.4. Demo: giao diện web

Link demo: https://diabetes-agent-seven.vercel.app/

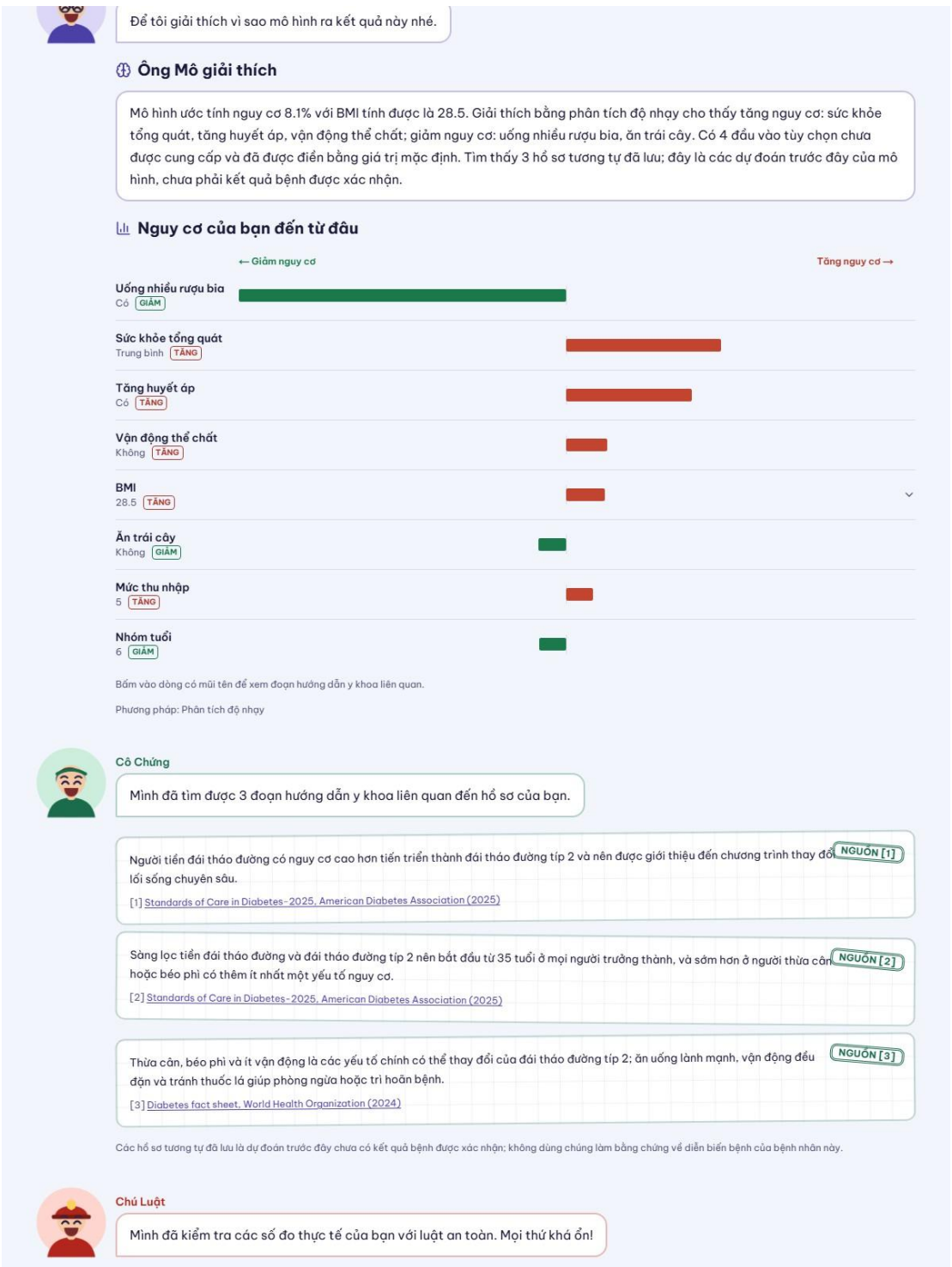
Giao diện web (Next.js) dẫn người dùng qua các câu hỏi ngắn rồi trình bày kết quả. Mỗi nhóm thành phần phía API được thể hiện bằng một "nhân vật" để người xem biết *ai làm gì*: **Bé Tiếp** (tiếp đón, hỏi hồ sơ), **Ông Mô** (mô hình dự đoán, giải thích), **Bà Nhớ** (bộ nhớ hồ sơ tương tự), **Cô Chứng** (truy xuất bằng chứng), **Chú Luật** (luật an toàn) và **Bạn Gợi** (khuyến nghị, *what-if*, policy). Phần cuối trang hiển thị `trace` của lượt xử lý dưới dạng dòng thời gian các bước; khi các thành phần bất đồng, cuộc trao đổi giữa các nhân vật xen vào ngay sau bước sinh ra nó (ảnh chụp dưới đây là bản giao diện trước khi thêm phần hội thoại). Hình 5.1 là màn nhập hồ sơ, Hình 5.2–5.5 là trang kết quả của hồ sơ mẫu (nữ 45 tuổi, 170 cm, 82,4 kg, huyết áp tâm thu 130 mmHg).



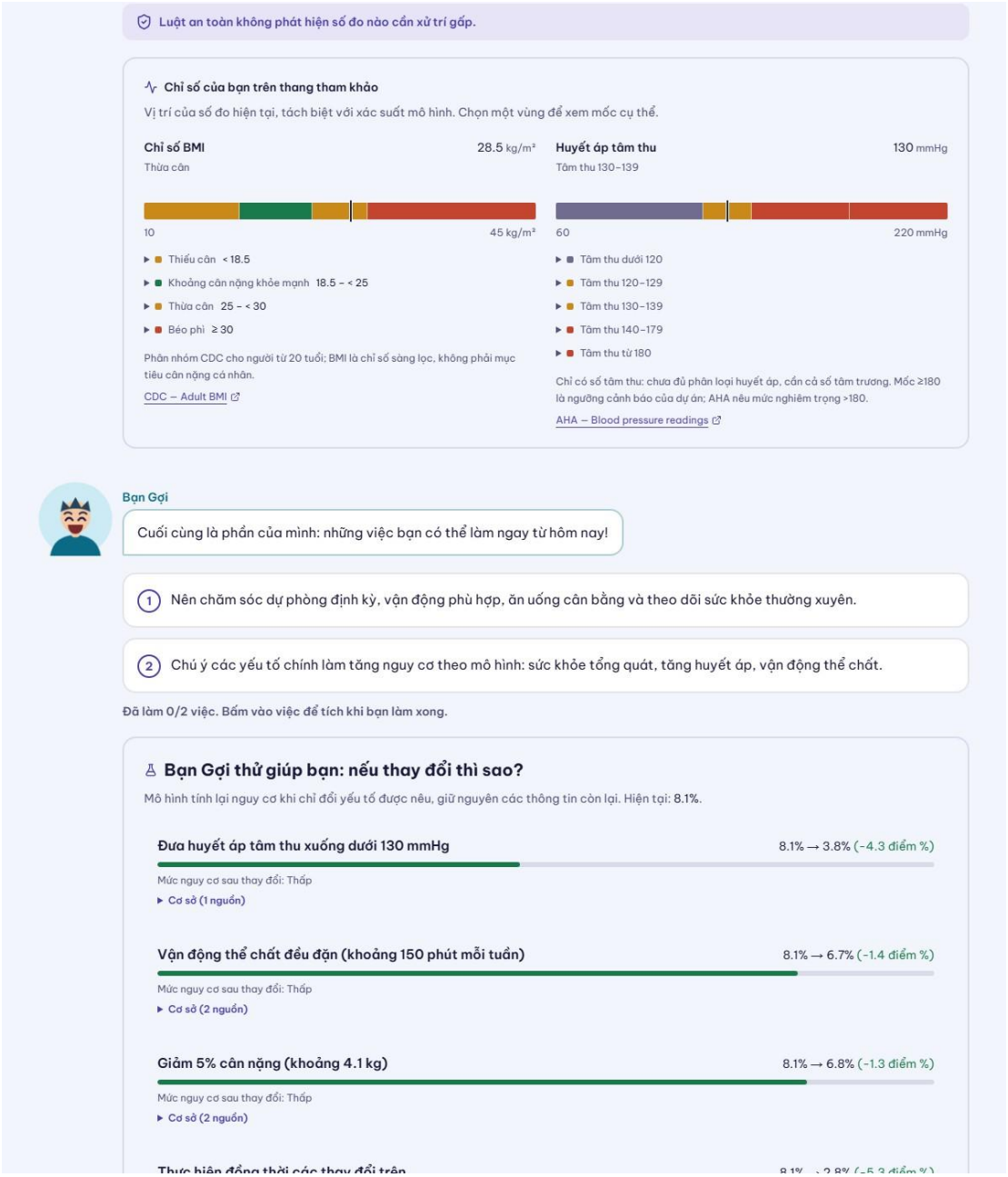
Hình 5.1. Màn nhập hồ sơ: Bé Tiếp hỏi số đo cơ bản và tự tính BMI



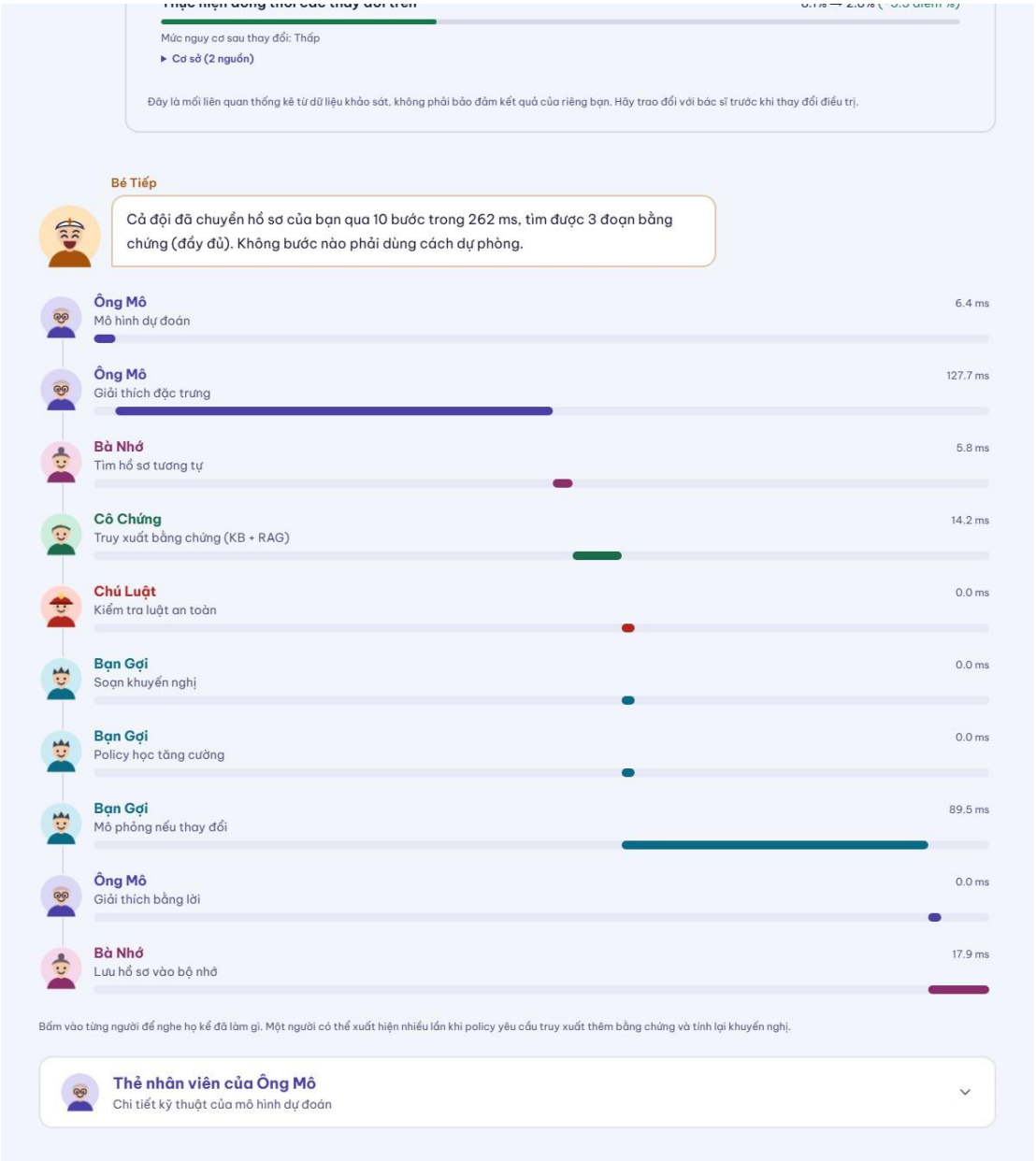
Hình 5.2. Kết quả: mức nguy cơ, hướng xử trí và câu hỏi bổ sung theo nhóm



Hình 5.3. Ông Mô giải thích kết quả và biểu đồ tác động của đặc trưng



Hình 5.4. Cô Chứng trích dẫn bằng chứng; Chú Luật kiểm tra an toàn



Hình 5.5. Thang chỉ số tham khảo, khuyến nghị của Bạn Gợi và *trace* của lượt xử lý

5.5. Ví dụ ba lượt đánh giá

Ba hồ sơ dưới đây được chạy thật qua `POST/v1/predict` (không LLM, **KB đọc từ tệp JSON vì lần chạy này không bật Neo4j**). Cột cuối cho thấy hành vi của hệ thống, không phải đánh giá lâm sàng.

Bảng 5.2. Ba hồ sơ chạy thật qua API

	A: nam 58 tuổi, 170 cm, 90 kg, HA 150, glucose lúc đói 92	**B: nữ 45 tuổi, 160 cm, 55 kg, HA 118, chưa đo glucose**	**C: 50 tuổi, 172 cm, 80 kg, HA 185, glucose 320**
Xác suất MLP / mức	17,3% / Thấp	0,5% / Thấp	3,9% / Thấp
Luật kích hoạt	Không	Không	`GLU-300`, `BP-180`
Xung đột	Không	Không	`CF-PRED-LOW-MEASURED-HIGH`
`effective_risk`	Thấp	Thấp	**Cao**
Hướng xử trí	Theo dõi định kỳ	Theo dõi định kỳ	**Khám khẩn cấp**
Bằng chứng	3 đoạn (đầy đủ)	3 đoạn (đầy đủ)	3 đoạn (đầy đủ)
Bước policy	`recommend_routine`	`request_glucose`	`escalate_to_clinician`
Câu hỏi bổ sung	Hỏi tiền sử đột quỵ	Hỏi kết quả glucose	Không (ưu tiên chuyển tuyến)
What-if	2 kịch bản	0	Không áp dụng
Tổng thời gian	352 ms	279 ms	142 ms

Hồ sơ C minh họa nguyên tắc quan trọng nhất: mô hình chỉ thấy các yếu tố lối sống và cho xác suất 3,9% (Thấp), nhưng huyết áp 185 mmHg và glucose 320 mg/dL kích hoạt hai luật khẩn cấp, luật xung đột `CF-PRED-LOW-MEASURED-HIGH` nâng `effective_risk` lên Cao, và policy không được phép hạ mức chuyển tuyến. Lượt chạy này cũng sinh một cuộc trao đổi hai thông điệp trong `trace.dialogue`: `SafetyGuard` → `RiskModel`, `challenge` ("mô hình dự đoán nguy cơ thấp nhưng số đo nằm trong khoảng nguy hiểm; ưu tiên số đo thực tế hơn dự đoán"), và `RiskModel` → `SafetyGuard`, `accept` (chấp nhận nâng nguy cơ hiệu lực lên Cao); hồ sơ A và B không có bất đồng nên `dialogue` rỗng. Hồ sơ B cho thấy hệ thống không coi "chưa đo glucose" là dấu hiệu yên tâm mà hỏi bổ sung. Với hồ sơ A, ba đoạn bằng chứng truy xuất gồm hai đoạn ADA (tiến triển từ tiền ĐTĐ, tuổi bắt đầu sàng lọc) và một đoạn WHO, hai trong ba đoạn được đồ thị gắn với yếu tố nguy cơ của bệnh nhân (`graph+semantic`). Thời gian chủ yếu nằm ở giải thích độ nhạy (khoảng 110–130 ms) và mô phỏng *what-if* (khoảng 120 ms); các bước còn lại dưới 50 ms.

5.6. Kiểm thử

Bộ kiểm thử gồm **154 kiểm thử**, đều đạt trong lần chạy cuối (3 cảnh báo deprecation của SciPy), độ phủ **84%** (`pytest --cov=.`).

Bảng 5.3. Bộ kiểm thử tự động

Tệp	Nội dung
`test_api.py`	Các endpoint, kiểm tra đầu vào (thiếu BMI hoặc chiều cao/cân nặng trả 422), luồng trả lời câu hỏi bổ sung
`test_orchestrator.py`	Truyền kết quả giữa thành phần, cô lập lỗi, `degraded`
`test_rules.py`	Từng luật cơ sở và luật xung đột, quyền ưu tiên của số đo
`test_input_quality.py`	Lý do trừ điểm chất lượng dữ liệu khớp điểm số
`test_risk_model.py`, `test_risk_bands.py`, `test_mlp.py`	Ánh xạ huyết áp sang `HighBP`, mức nguy cơ theo artifact (cờ dương tính không bao giờ ở mức Thấp), MLP dự đoán được sau khi nạp mà không cần torch và hiệu chuẩn như các ứng viên khác
`test_retriever.py`, `test_neo4j_kb.py`	Truy xuất, trạng thái bằng chứng, nạp và truy vấn KB, dự phòng
`test_rl.py`	Môi trường, Q-learning, luật ghi đè policy
`test_dialogue.py`	Thông điệp giữa các thành phần: không bất đồng thì rỗng, `challenge` rồi `accept`/`propose`/`inform` đúng người nhận; đề xuất tìm mở rộng được thực hiện đúng một lần, đổi bằng chứng và khuyến nghị nhưng không hạ mức chuyển tuyến; tìm mở rộng lỗi thì giữ bằng chứng cũ và báo lỗi
`test_step11.py`, `test_patient_guidance.py`, `test_what_if.py`, `test_step12.py`	Giải thích không gán nhầm đặc trưng tự điền, điểm chất lượng theo tỉ lệ mục tự điền, hồ sơ tương tự theo khoảng cách số, thang chỉ số, câu hỏi bổ sung, mô phỏng *what-if*, kịch bản chấp nhận trên artifact thật

5.7. Đánh giá

Mô hình. Xem Bảng 4.1: MLP hiệu chuẩn đạt ROC-AUC 0,826 và AP 0,414 trên 50.736 hồ sơ kiểm tra, ngang các mô hình boosting. Phân tầng có ý nghĩa: nhãn 1 chiếm 7,1%, 32,1% và 49,0% lần lượt ở mức Thấp, Trung bình và Cao.

Policy RL. Xem Bảng 4.5; chỉ có giá trị trong môi trường mô phỏng.

Tích hợp. Lần chạy demo tích hợp ngày 03/11/2026 (`scripts/demo_step12.py --require-neo4j --with-web`) đạt **26/26** kịch bản trên Neo4j thật (`kb_source=KnowledgeGraph`): hồ sơ đủ và thiếu dữ liệu, cảnh báo, thiếu bằng chứng, lỗi từng thành phần (retriever, memory, explainer, recommender, safety, policy), lỗi đầu vào (422), lịch sử, và cả ba proxy web khi back-end ngừng (HTTP 502). Phần luồng hỏi bổ sung được kiểm tra bằng trình duyệt ở hai kích thước (1440×1000 và 390×844) với các kiểm tra như không hỏi lặp, không gán giá trị giả, không tràn bố cục. Lần chạy demo dùng mẫu câu dự phòng, chưa gọi OpenAI thật.

Điều chưa được đánh giá. Chưa có đánh giá đầu–cuối trên ca bệnh thật (độ nhạy/đặc hiệu của cả hệ thống), chưa có benchmark truy xuất, chưa có thử nghiệm người dùng, và các số đo độ trễ ở Bảng 5.2 chỉ là ba lần chạy, không phải thống kê.

5.8. Hạn chế và hướng phát triển

- **Mô hình**: dữ liệu BRFSS 2015 chỉ của Hoa Kỳ, nhãn tự khai và gộp tiền ĐTĐ với ĐTĐ; chưa hiệu chỉnh cho người Việt; MLP không hơn rõ rệt các mô hình boosting. `HighBP` suy ra từ một số đo huyết áp là xấp xỉ.
- **Glucose không vào mô hình**: xác suất MLP không đổi khi bổ sung glucose; glucose chỉ tác động qua luật và tư vấn.
- **KB/RAG**: chỉ 21 đoạn bằng chứng từ 5 nguồn, chưa thẩm định chuyên môn; chưa có bộ đánh giá truy xuất; embedding MiniLM chủ yếu cho tiếng Anh. Hướng sửa: embedding đa ngôn ngữ, mở rộng KB, xây bộ câu hỏi có đáp án.
- **Luật**: ngưỡng là của dự án, chưa được bác sĩ thẩm định.
- **RL**: môi trường và phần thưởng tự đặt, chưa học từ phản hồi thật; không chứng minh hơn bảng kiểm cố định.
- **Giao tiếp agent–agent còn tối thiểu**: các thành phần vẫn do Orchestrator gọi tuần tự. Phần phản biện (`challenge`/`accept`/`propose`/`inform`) dùng hàm phản hồi tất định; chỉ đề xuất tìm mở rộng bằng chứng của retriever được thực hiện thật, còn lại chỉ ghi lại bất đồng do luật xác định. Chưa có thương lượng nhiều vòng, chưa có agent nào tự khởi xướng hay thay đổi quyết định lâm sàng cuối, và đề xuất tìm mở rộng chưa kích hoạt trên hồ sơ thật vì bằng chứng của KB hiện tại luôn đủ. Lần chạy demo 26/26 thực hiện trước khi thêm phần này và chưa được chạy lại.
- **Chưa có thử nghiệm với người dùng thật.** Mọi kết quả chỉ nhằm mục đích học tập, không thay thế tư vấn y khoa.

KẾT LUẬN

Dự án đã hiện thực hóa sơ đồ hệ đa agent cho bệnh tiểu đường thành một hệ thống chạy được đầu–cuối. Dữ liệu BRFSS 2015 (253.680 hồ sơ) đi qua Deep Learning để dự đoán và phân loại nguy cơ: MLP PyTorch hiệu chuẩn đạt ROC-AUC 0,826 trên tập kiểm tra tách riêng, ngang các mô hình boosting, với ngưỡng và ba mức nguy cơ chọn trên tập kiểm định. Tri thức từ ADA, WHO, CDC, NIDDK và thử nghiệm DPP được tổ chức thành đồ thị Neo4j (20 thực thể, 35 quan hệ, 21 đoạn bằng chứng) và truy xuất bằng RAG kết hợp liên kết đồ thị, để khuyến nghị luôn kèm nguồn. Bộ luật 9 luật bảo đảm số đo thực tế thắng dự đoán thống kê, và policy Q-learning chọn bước tiếp theo luôn nằm dưới quyền ưu tiên của luật. Khi các thành phần bất đồng, chúng trao đổi thông điệp có kiểu (phản biện, chấp nhận, đề xuất, thông báo) được ghi vào `trace`. Hệ thống được kiểm thử bằng 154 kiểm thử tự động (độ phủ 84%) và demo tích hợp 26/26 kịch bản trên Neo4j thật.

Bài học lớn nhất của dự án là trong bài toán y tế, giá trị của kiến trúc nhiều thành phần nằm ở chỗ mô hình thống kê, bằng chứng có nguồn và luật tường minh *kiểm tra chéo* nhau, mọi kết quả truy vết được tới một bước cụ thể (`trace`), và phần học luôn đặt dưới sự giám sát của tri thức tường minh. Đồng thời, báo cáo nêu rõ các giới hạn: dữ liệu và nhãn của Hoa Kỳ, MLP không vượt trội, RL chỉ trên môi trường mô phỏng, chưa có benchmark truy xuất và chưa thẩm định chuyên môn. Hướng phát triển tiếp theo gồm: hiệu chỉnh mô hình trên dữ liệu người Việt, dùng embedding đa ngôn ngữ và xây bộ đánh giá truy xuất, nhờ chuyên gia y khoa thẩm định KB và ngưỡng luật, và mở rộng giao tiếp giữa các agent từ ghi nhận bất đồng sang thương lượng nhiều vòng.

Tài liệu tham khảo

[1] American Diabetes Association Professional Practice Committee, "Standards of care in diabetes—2025," *Diabetes Care*, vol. 48, no. Suppl. 1, 2025. https://diabetesjournals.org/care/issue/48/Supplement_1

[2] Diabetes Prevention Program Research Group, "Reduction in the incidence of type 2 diabetes with lifestyle intervention or metformin," *New England Journal of Medicine*, vol. 346, no. 6, pp. 393–403, 2002.

[3] World Health Organization, *Diabetes fact sheet*, 2024. https://www.who.int/news-room/fact-sheets/detail/diabetes

[4] Centers for Disease Control and Prevention, *Diabetes risk factors and prevention*, 2024. https://www.cdc.gov/diabetes/

[5] National Institute of Diabetes and Digestive and Kidney Diseases, *Diabetes information*, 2024. https://www.niddk.nih.gov/health-information/diabetes

[6] P. K. Whelton *et al.*, "2017 ACC/AHA guideline for the prevention, detection, evaluation, and management of high blood pressure in adults," *Hypertension*, vol. 71, no. 6, pp. e13–e115, 2018.

[7] Centers for Disease Control and Prevention, *CDC Diabetes Health Indicators* (BRFSS 2015), UCI Machine Learning Repository, id 891. https://archive.ics.uci.edu/dataset/891/cdc+diabetes+health+indicators

[8] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.

[9] N. R. Jennings, K. Sycara, and M. Wooldridge, "A roadmap of agent research and development," *Autonomous Agents and Multi-Agent Systems*, vol. 1, pp. 7–38, 1998.

[10] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[11] Foundation for Intelligent Physical Agents, *FIPA ACL message structure specification* (SC00061G), 2002.

[12] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[13] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *EMNLP-IJCNLP*, 2019.

[14] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[15] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.

[16] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[17] Neo4j, Inc., *Neo4j graph database and the Cypher query language*. https://neo4j.com/docs/

[18] Chroma, *Chroma: the open-source embedding database*. https://docs.trychroma.com/